/tikz/,/tikz/graphs/
conversions/canvas coordinate/.code=1 , conversions/coordinate/.code=1

circular

Internship Report

First Year of the Master of Computer Science – Cyber-Physical Systems Specialization

Automated Consistency Verification for Replicated Data Systems

Isabelle Coget

Submitted to:
Institut Polytechnique de Paris

Internship conducted at:
Laboratoire i3s, Université Côte d'Azur

January 27, 2026

Abstract

As distributed systems grow increasingly complex, verifying the reliability of their behavior and program correctness becomes essential. Verification provides formal guarantees that a program behaves as intended under specific system assumptions.

In this work, we define a formal framework for consistency verification in replicated data systems and identify its key constraints. Within this context, we prove that the consistency verification problem is decidable. We also propose a method to encode execution traces as finite words, enabling the concrete verification of trace properties. This approach enables verification of consistency for any execution trace over finite data domains.

Contents

1. Introduction

Verification is a significant phase of software development, aimed at ensuring that a program behaves as intended. It highlights unexpected and undesired behaviors an execution can present, such as memory leaks, bugs and crashes. Notable failures that could have been prevented through proper verification include the Ariane 5 crash in 1996 and the software failure of the Bank of New York in 1985. Fortunately, advances in modern verification techniques enable early identification of such critical faults, preventing similar disasters from occurring. As systems become more complex and decentralized, robust program verification tools are essential to ensure correctness, particularly for replicated objects and distributed systems, on which this paper is focused. Distributed systems employ different communication models, such as message-passing and replicated object programs. The verification of the former focuses on the communication model itself, while the latter relies on consistency models.

Message-passing communication models have been previously studied [?] by means of message sequence charts. These charts keep track of message exchanges between agents throughout an execution, representing the different possible behaviors of the program. A message-passing program is said to be sound with respect to a property if all of its executions verify it. That is, the program is considered to have failed verification if even one execution violates the property. The latter may be expressed as a logical formula, thereby reducing verification to the satisfiability problem of monadic second-order (MSO) formulas over message sequence charts. Therefore, to ensure the decidability of verification tasks involving message sequence charts, it is necessary to enforce certain restrictions, such as synchronizability or bounded buffer sizes [?] .

In contrast to message-passing programs, agents in replicated data systems communicate by updating data rather than through message exchange. All agents maintain replicas of the same data, but because updates propagate with uncertainty and delays, the copies may not be synchronized and can become inconsistent across different agents. Consitency models define restrictions on how these data updates are propagated accross the different replicas. Each consistency model provides a distinct level of guarantee regarding the visibility and ordering of updates in the system.

Just as message sequence charts track message exchanges in message-based systems, *histories* are used to record data updates in replicated data systems [?] . A *history* is defined as the set of all data updates that occur during a single execution of a program. These updates are referred to as *operations*. This paper considers only read and write operations, as many core actions in replicated data systems can be expressed using these primitives. An operation's attributes include its start and return times, input and output values, target object, and performed action. The operations within a history must collectively satisfy certain consistency models (linearizability, eventual consistency, read-my-writes), which are expressed as logical formulas over operations [?] .

However, the logical formulas defined in [?] do not correspond to any automated logical system (such as propositional, first-order, or second-order logic) and must therefore be reformulated within a more rigorous framework. Thus, we define the *logic over histories* in terms of MSO logic, the target formalism for this work. Our choice to restrict the logic to MSO is motivated by the following fundamental result, known as Courcelle's Theorem [?] : every graph property expressible in the monadic second order logic of graphs is decidable in linear time on graphs of bounded tree-width. Intuitively, *tree-width* is a measure of how close a graph is from being a tree, where a tree is a graph that contains no cycles[1]. The formal definiton of tree-width is provided in Appendix.

The first part of this report demonstrates how histories can be encoded as graphs of bounded tree-width, which, by Courcelle's theroem, ensures that model checking over these histories is decidable and can be performed in linear time. However, model checking is performed on a single history, while program verification requires that the specified properties hold over all possible histories, which

[1]A cycle is a non-empty path in which the first and last vertices are the same

considerably hardens the problem. Courcelle's Theorem is traditionally stated in the context of model checking; thus, to fit more naturally into the verification framework, it can be reformulated as follows: the satisfiability of an MSO property over k-bounded graphs[2] is decidable (although of non-elementary complexity[3]).

To the best of my knowledge, no solver supports the entirety of MSO logic. Existing solvers for restricted fragments include MiniSAT [**?**] and Glucose [**?**], which operate over propositional logic. This logic is considerably less expressive than required by our approach and cannot handle the types of structures in our setting. In contrast, MONA[**?**] is an MSO solver designed for specific decidable fragments[4], translating such formulas into finite-state automata to enable satisfiability and model checking on the resulting automaton. In the second part of this paper, we show how MONA can be applied to the MSO satisfiability problem over histories of replicated-object programs, and how these histories can be encoded to meet MONA's logical restrictions.

Overall, this leads us to the central question:

> **What restrictions are required to ensure the decidability of MSO satisfiability over replicated-object histories, and how can consistency verification be automated under these constraints?**

2. Histories

Tracking the sequence of computed actions in replicated data systems can be achieved through various methods. In this paper, we base our approach on the notion of histories defined in [**?**] . A history of a program execution is defined as a set of *operations* that represent all the actions performed during the computation. Histories can be visualized as timelines, where each process is represented by a horizontal line aligned to a global clock. The execution of an operation is shown as a solid segment, while dashed segments indicate periods when the process is idle. These timelines are read from left to right. Figure **??** illustrates this graphic representation. Each operation has the following attributes: the process launching the operation, the invocation time, the return time, the type of operation, the object on which the operation takes effect, the input value of the operation and the output value of the operation. For formalization purposes, it is essential to explicitly define the domain to which each attribute belongs. Since our objective is to encode these histories for analysis using MONA, we introduce certain constraints on these sets of attributes. In particular, MONA does not support infinite sets; hence, certain sets of attributes must be finite. Without this finiteness assumption, the encodings presented in this paper would not be computable.

Thereby we define

* $\mathbb{P}$ the set of processes from which operations are launched
* $\mathbb{T}$ the set of types of operations
* $\mathbb{O}$ the set of objects, which are the data to be modified or read by operations (objects can be registers or variables for instance)
* $\mathbb{V} \cup \{\nabla, \Phi\}$ where $\mathbb{V}$ is the set of values that can be read or written on objects, and Φ represents a non-relevant or not needed value (for instance, we can claim that for a write operation, the input value is the value to be written, and the output value is Φ; as a write operation does not necessarily

[2] A graph is k-bounded if it has a structural property bounded by k

[3] A problem has non-elementary complexity in n if it grows faster than $\left.2^{2^{\cdot^{\cdot^{\cdot^{2^n}}}}}\right\} k$ for any k finite

[4] MONA supports formulas in the Weak Monadic Second-Order Logic of One Successor (WS1S) and Two Successors (WS2S), which are decidable fragments of MSO logic over finite words and trees, respectively.

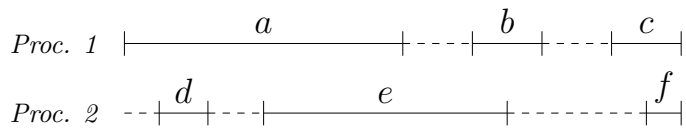


Figure 1: *A history on two processes, with operations a, b, c, d, e and f*

need to return anything), and ∇ stands for the "output value" of an operation that never returns (concretely, ∇ represents the absence of a return value due to a never ending operation)

and $\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V}$ finite.

Invocation and return times can be defined over either absolute or relative time, and are totally ordered. These times are defined in $\mathbb{R}^+ \cup \{+\infty\}$. We include $+\infty$ to represent operations that never return. Assigning $+\infty$ as a return time reflects that, beyond a certain point, the operation is considered to never terminate. Since this threshold is proper to each system, we will keep the notation $+\infty$ throughout this paper, and will allow return times to take the value $+\infty$.

We recall the attributes from [**?**] along with their respective domains. Throughout this paper, we adopt the object-oriented notation, namely *operation.attribute*.

* $proc \in \mathbb{P}$ process on which the operation is executed
* $stime \in \mathbb{R}^+$ invocation time
* $rtime \in \mathbb{R}^+ \cup \{+\infty\}$ return time, $+\infty$ if the operation never returns (we note that for any operation a we have $a.stime < a.rtime$)
* $type \in \mathbb{T} = \{read, write\}$ type of operation, here either *read* or *write*
* $obj \in \mathbb{O}$ object on which the operation takes effect
* $ival \in \mathbb{V}$ input value, Φ if the operation is a *read*
* $oval \in \mathbb{V}$ output value, Φ if the operation is a write, and ∇ if the operation does not return

Here read operations are assumed to have no input value, which we denote by Φ. Similarly, since write operations do not return a value, their output is also set to Φ.

We define the set of all finite execution histories as follows. The symbol $\wedge$ denotes logical conjunction, which can be read as "and".

Definition 2.1 (Execution History Set)**.** Let $\mathbb{P}$, $\mathbb{T} = \{read, write\}$, $\mathbb{O}$, and $\mathbb{V}$ be finite sets representing, respectively, processes, operation types, objects, and values. We define $\mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ as the set of all well-defined finite histories of executions, where attributes are drawn from these sets. A history is considered well-defined if every operation has the appropriate attributes and if, for each process, operations do not overlap (a process executes one operation at a time, it launches a new execution only if the previous one has returned). Formally, this means

$$\forall H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V}).\ |H| < +\infty \wedge$$
$$\forall a \in H.\ \Big(a.proc \in \mathbb{P} \ \wedge\ a.stime \in \mathbb{R}^+ \ \wedge\ a.rtime \in \mathbb{R}^+ \ \wedge\ a.stime < a.rtime \wedge$$
$$a.type \in \mathbb{T} \ \wedge\ a.obj \in \mathbb{O} \ \wedge\ a.ival \in \mathbb{V} \ \wedge\ a.oval \in \mathbb{V} \wedge$$
$$(a.type = read \Rightarrow a.ival = \Phi) \ \wedge\ (a.type = write \Rightarrow a.oval = \Phi) \ \wedge\ (a.rtime = +\infty \Leftrightarrow a.oval = \nabla) \Big) \wedge$$
$$\forall b \in H.\ \forall c \in H.\ b.proc = c.proc \Rightarrow (b.rtime < c.stime \ \vee\ c.rtime < b.stime)$$

All operations can be compared based on their attributes, meaning that relations can be defined between operations. A relation is a property that involves one to a few operations; in our case, relations are binary — that is, they relate pairs of operations. Among the relations defined by [**?**] , we primarily use the *returns-before* and the *same-session* relation. For a history $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and operations $a, b \in H$, the returns-before relation denoted as $a \xrightarrow{rb} b$, means that a returns before b starts; formally, $a.rtime < b.stime$. As for the same-session relation denoted as $a \xrightarrow{ss} b$, means that a and b are launched on the same process; formally, $a.proc = b.proc$.

The $\xrightarrow{rb}$ relation will constitute the foundation of our graph representation of histories; however, we will also need to introduce a more restrictive relation, which we call *direct succession*.

Definition 2.2 (Direct Successor)**.** Let $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and $a, b \in H$. We say that b is the direct successor of a on process p, written $a \overset{p}{\rightharpoonup} b$, if b is the first operation to be launched on process p after a returns.
Formally, for $p \in \mathbb{P}$:

$$a \overset{p}{\rightharpoonup} b \quad \Leftrightarrow \quad b.proc = p \ \wedge \ \neg\exists c.\Big(c \in H \wedge c \neq b \wedge c \xrightarrow{ss} b \wedge a \xrightarrow{rb} c \wedge c \xrightarrow{rb} b\Big)$$

The set of all such direct successors of a is written as $\sigma(a)$ and defined as

$$\sigma(a) = \bigcup_{p \in \mathbb{P}} \{b \mid b \in H \wedge a \overset{p}{\rightharpoonup} b\} \quad \subset H$$

Additionally, we say that b is a direct successor of a, written $a \rightharpoonup b$, if $b \in \sigma(a)$.
Note that the relation $a \overset{p}{\rightharpoonup} b$ holds information about the process of b (namely, p), but does not reveal any information about the process on which a is executed.

We note that for all $a \in H$, $|\sigma(a)| \leq |\mathbb{P}|$, because any operation has at most one direct successor on each process.

Now that histories and their associated relations are well defined, we can proceed to study how to encode these histories as graphs; in particular, as a graphs of bounded tree-width.

3. Graphs of histories

A history of a program execution can be perceived as a timeline gathering different data updates. Key information held in a timeline includes when operations start and return, and the order in which they are performed. Since the *returns-before* relation holds exactly this information, it will serve as the foundation of our graphs. Graphs are data structures that relate some elements (referred to as *vertices* or *nodes*) according to some relation represented by *edges*. In this work, we focus on graphs that are restricted to edges having exactly two end-points. Since edges are intended to represent the *returns-before* relation, the *vertices* of these graphs correspond precisely to the operations in a history.

The usual notation for a graph G is $G = (V, E)$ with V the set of vertices and E the set of edges. As an edge associates two vertices, the set E is a subset of $V \times V$; i.e. an edge is a couple (a, b) with $a, b \in V$. The set of edges in a graph can be represented either as a set of pairs or as a binary relation over the set of vertices. For instance, if a graph $G = (V, E)$ is such that $(a, b) \in E$ *if and only if* $a \xrightarrow{rb} b$; then G can equivalently be defined as $G = (V, \xrightarrow{rb})$.

Although the *returns-before* relation denotes the desired information for our graphs, it is overly dense, inducing too many edges. For a history $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ of arbitrary size, encoding H on a graph $G = (H, \xrightarrow{rb})$ results in G being a clique — a graph in which every pair of distinct vertices is

connected by an edge. We recall the objective of this section: to encode any history $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ as a graph of bounded tree-width. However, cliques of arbitrary size are not of bounded tree-width. Thus, the $\xrightarrow{rb}$ relation is not sufficiently refined for our study.

It is the transitivity [5] of $\xrightarrow{rb}$ that makes it too dense. Indeed, if there are operations such that $a \xrightarrow{rb} b$, $b \xrightarrow{rb} c$ and $a \xrightarrow{rb} c$, the edge $a \xrightarrow{rb} c$ in unnecessary as having $a \xrightarrow{rb} b$ and $b \xrightarrow{rb} c$ naturally implies $a \xrightarrow{rb} c$. When a transitive relation is encoded on a graph, its transitivity is represented by *paths* in the graph. Intuitively, a path between two vertices is a sequence of edges that allows traversal from the first vertex to the second. Formally, for a binary relation $\rightsquigarrow$ over the set V, and a graph $G = (V, \rightsquigarrow)$; a path between $a \in V$ and $b \in V$ is a sequence of vertices $(v_1, \cdots, v_n), n \geq 0$, such that

$$a = v_1 \rightsquigarrow \cdots \rightsquigarrow v_n = b.$$

Since transitivity corresponds to the existence of a path, we aim to define a new binary relation $\rightarrow$ on operations, such that its transitive closure [6] is the $\xrightarrow{rb}$ relation; and such that $G = (H, \rightarrow)$ is of bounded tree-width. Defining a graph of history using $\rightarrow$ significantly reduces the number of edges, while maintaining the "timeline" structure encoded by the *returns-before* relation.

We use the notation $(H, \rightarrow)$ to denote a graph whose vertices are the operations in H, and whose edges are defined by the relation $\rightarrow$. We write $a \twoheadrightarrow b$ to indicate that there exists a path from a to b in the graph.

We now define the relation $\rightarrow$ using an algorithm. This choice is motivated by the need to minimize the number of edges that $\rightarrow$ induces in our graphs, which requires conditional logic (such as *if-else* statements) only available when using algorithms. While it might be theoretically possible to define $\rightarrow$ without resorting to an algorithm (e.g., through unions or restrictions of relations over operations), proving that the resulting graph has bounded tree-width would likely be significantly more difficult. Therefore, in this paper, we do not provide a formal definition of $\rightarrow$ outside of its algorithmic construction.

In this algorithmic construction, our goal is to reduce the edge set while preserving the necessary information. Specifically, if $a \xrightarrow{rb} b$, we require that this relationship can still be recovered from the graph induced by $\rightarrow$.

We define a function *ord* that takes as input a history, and returns a tuple of ordered operations of the history.

Definition 3.1 (Ordering Function)**.** Let $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$.

$$\begin{gathered} ord(H) \coloneqq (a_1, ..., a_n) \text{ such that } \{a_1, ..., a_n\} = H \\ \text{and for all } a_i, a_j \text{ with } i < j\text{: } a_i.stime \leq a_j.stime \end{gathered}$$

The algorithm operates as follows. It takes as input $ord(H)$, where $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$, and traverses $ord(H)$ in reverse, starting from the last element and proceeding to the first. For each operation, its direct successors are extracted, sorted according to *ord*, and then processed one by one. If a path to a given successor already exists in the graph, no action is taken; otherwise, a direct edge to that successor is added. This procedure ensures that the number of edges is minimized.

The algorithm is formalized as follows.

[5] A relation R is said to be transitive when xRy∧yRz⇒xRz

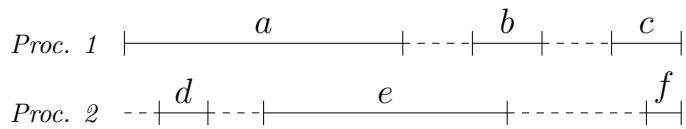
[6] The transitive closure of a relation R is the smallest relation that contains R and is transitive. The transitive closure of R is traditionally denoted by R^+

Algorithm 1 Building a graph of history

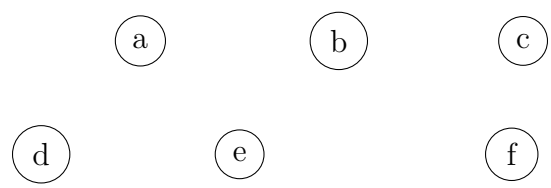
$H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ with $ord(H) = (a_1, ..., a_n)$, $n \in \mathbb{N}$ $G = (H, \rightarrow)$ $i \leftarrow n - 1$ $E \leftarrow \emptyset$ $G \leftarrow (H, E)$
$i > 0$ $S \leftarrow (b_1, ..., b_m) = ord(\sigma(a_i))$ $j \in \{1, ..., m\}$ $\neg(a_i \twoheadrightarrow b_m)$ $E \leftarrow E \cup \{(a_i, b_m)\}$ $G \leftarrow (H, E)$
$i \leftarrow i - 1$

> **Remark 3.1.** When building a directed graph as above, each directed edge can be interpreted as a "jump forward in time" — that is, traversing an edge corresponds to moving along the timeline of certain operations in the history.

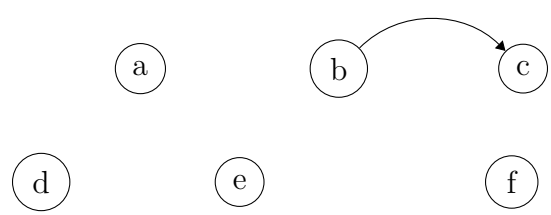
Example 1. As an example, we now apply this algorithm to the history depicted in Figure **??**, which is reproduced below for convenience.



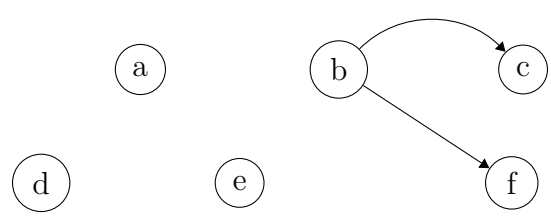
Let $G = (H, \rightarrow)$ be the graph of history of $H = \{a, b, c, d, e, f\}$ as represented above. The set of processes is $\mathbb{P} = \{p_1, p_2\}$, where the line labeled *Proc.1* corresponds to p_1, and the line below to p_2. The operations are ordered by $ord(H) = (a, d, e, b, c, f)$. For now, $\rightarrow$ does not relate any operations; that is, at the beginning of the algorithm, G is defined as $(H, \emptyset)$. At this stage, G is as follows.



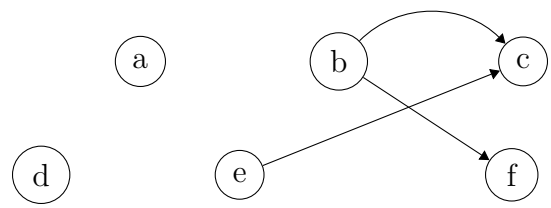
The first operation processed is f, which has no direct successor. As a result, nothing is done. Next comes c, which also has no direct successors, so again, no action is taken. Then, b is processed. Its ordered successors are (c, f). There is no existing path from b to c, thus the edge $b \rightarrow c$ is added.



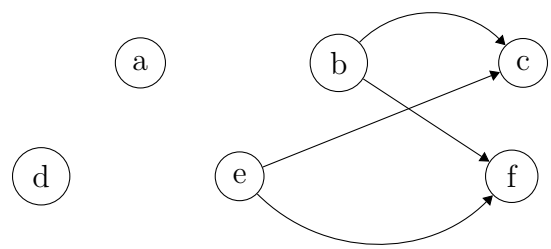
Similarly, as no path exists from b to f, the edge $b \rightarrow f$ is added as well.



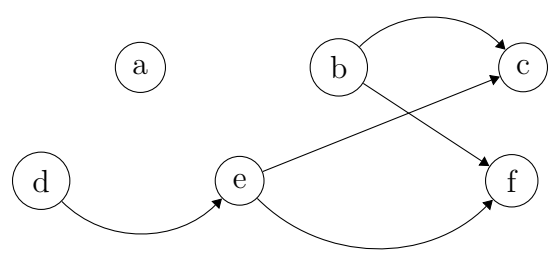
Now is considered e, which has ordered successors (c, f). Since no path exists from e to c, we add the edge $e \to c$.



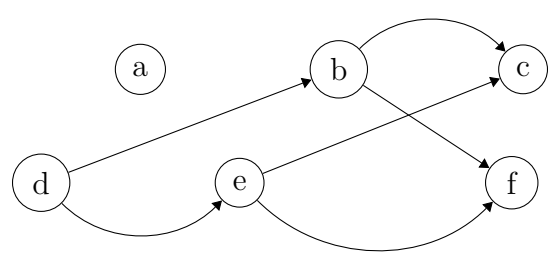
Likewise, as there is no existing path from e to f, the edge $e \to f$ is added.



Continuing along our history, d has successors (e, b). Since no path exists from d to e, we add the edge $d \to e$.



Likewise, as there is no existing path from d to b, the edge $d \to b$ is added.



Finally, a has successors (b, f). Since there is no path between a and b the needed edge $a \to b$ is added.

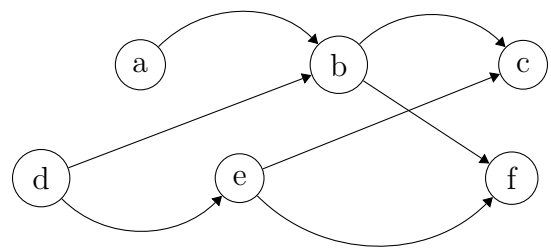


Figure 2: *The graph of history of* **??**.

However, a path now exists between a and f, so no additional edge is added. The graph $G = (H, \rightarrow)$ is now complete.

$\Diamond$

We denote by $Alg1(H)$ the graph of history H computed by Algorithm **??**. Let $\mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$ denote the set of all such graphs for histories in $\mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ with $|\mathbb{P}| = m$. Formally, this set is defined as follows.

Definition 3.2 (Execution Graph Set)**.** For $m \in \mathbb{N}$,

$$\mathcal{G}^m_{\mathcal{A}lg1} := \left\{ \mathit{Alg1}(H) \ \mid \ H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V}) \text{ and } |\mathbb{P}| = m \right\}$$

Let $G = (H, \rightarrow) \in \mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$. Although for $a, b \in H$, the existence of an edge $a \rightarrow b$ imples that b is a direct successor of a, the converse does not necessarily hold. This can be easily seen in the history shown in **??** and the corresponding graph in **??**. Operation d is a direct successor of c, yet there is no edge between the corresponding nodes in the graph. This property is significant, as it allows us to reduce the number of edges and consequently obtain a tighter bound on the tree-width.

Note Appendix **??** lists all relations and functions defined earlier.

4. Tree-width boundedness

[**?**] introduces multiple upper bounds on tree-width, such as *cutwidth*, which is the focus of this work. Among the proposed parameters, cutwidth provides the most intuitive interpretation for the types of graphs considered here.

This section is dedicated to showing that the set $\mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$ has uniformly bounded tree-width, meaning there exists an integer that bounds the tree-width of every graph in $\mathcal{G}^m_{\mathcal{A}lg1}$.

The cutwidth of an undirected graph G is the minimum $k \in \mathbb{N}$ such that for some linear ordering of the vertices of G along a horizontal axis, every vertical cut intersects at most k edges. Using the cutwidth is advantageous in our work, as vertices of graphs of history already represent a timeline, which is a linear ordering. To rigouroulsy define the cutwidth of a graph, it is necessary to introduce the notion of *cut* of a graph.

Definition 4.1 (Graph Cut)**.** Let $G = (V, E)$ be a graph, let $\mathscr{L}(G)$ denote the set of all possible linear orderings of the vertices of G. For $n = |V|$, $\ell \in \mathbb{N}$ and $f = (v_1, ..., v_n) \in \mathscr{L}(G)$,

$$Cut_\ell(f, G) := (\{v_1, ..., v_\ell\}, \{v_{\ell+1}, ..., v_n\})$$

defines a bipartition of G, and is referred to as a *cut of* G. An edge (u, v) crosses a cut $(L, R) := Cut_\ell(f, G)$, if $(u, v) \in L \times R$ or $(u, v) \in R \times L$.

Formally, *cutwidth* is defined as follows.

Definition 4.2 (Cutwidth)**.** For $G = (V, E)$ a graph, let $\mathscr{L}(G)$ denote the set of all possible linear orderings of the vertices of G. For $n = |V|$ and $\ell \in \mathbb{N}$,

$$cutwidth(G) = \min_{f \in \mathscr{L}(G)} \max_{1 \le \ell \le n} \left| \left\{ (v_i, v_j) \in E \;\middle|\; i \le \ell < j \text{ or } j \le \ell < i \text{ , and } (v_1, ..., v_n) = f \right\} \right|.$$

Equivalently, using the notion of graph cut,

$$cutwidth(G) = \min_{f \in \mathscr{L}(G)} \max_{1 \le \ell \le n} \left| \left\{ (u, v) \;\middle|\; (u, v) \in L \times R \text{ or } (v, u) \in L \times R, \text{ for } (L, R) := Cut_\ell(f, G) \right\} \right|.$$

In this work, the cutwidth will be bounded using the linear ordering function *ord* defined earlier.

We now introduce the necessary results to establish that $\mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$ has uniformly bounded cutwidth.

4.1. Preliminary results

It is worth noting that both $\xrightarrow{rb}$ and $\twoheadrightarrow$ are transitive. Indeed, given $a \xrightarrow{rb} b$ and $b \xrightarrow{rb} c$, it follows naturally that $a \xrightarrow{rb} c$. Similarly, for $\twoheadrightarrow$, if $d \twoheadrightarrow e$ and $e \twoheadrightarrow f$, then $d \twoheadrightarrow f$.

For a a vertex of a graph, we denote its out-degree as $deg_{out}(a)$, and its in-degree as $deg_{in}(a)$. The *out-degree* of a vertex is the number of edges connected to it that are directed away from the vertex. The *in-degree* is the number of connected edges directed towards the vertex.

Lemma 4.1. Let $G = (H, \rightarrow) \in \mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$. Then, for all $a \in H$, $deg_{out}(a) \le m$.

Proof. In Algorithm **??**, line 5 marks the beginning of the iteration over all operations in $ord(H)$. Let a denote the operation currently being processed in the algorithm. Between lines 7 and 12, the algorithm may add up to m out-going edges from a, since by definition $\sigma(a) \le |\mathbb{P}| = m$.

As each vertex has its out-going edges built as above, all vertices of G have an out-going degree of at most m.

□

We now present two propositions that will support the proof of Lemma **??**.

Proposition 4.2. Let $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ be a history. For $a, b \in H$,

$$a \xrightarrow{rb} b \Leftrightarrow \text{there exists a tuple } (h_1, ..., h_q),\ h_i \in H, q \in \mathbb{N} \text{ such that } a \overset{p}{\rightharpoonup} h_1 \overset{p}{\rightharpoonup} \dots \overset{p}{\rightharpoonup} h_q \overset{p}{\rightharpoonup} b \text{ with } p := b.proc.$$

Proof. $\Rightarrow$)

Let $a, b \in H$ with $a \xrightarrow{rb} b$, and let T denote the tuple we aim to construct. We distinguish two main cases.

* if b is a direct successor of a; that is, there exists $p \in \mathbb{P}$ such that $a \xrightarrow{p} b$, then we set $T = ()$
* if b is not a direct successor of a, we proceed as follows.

Let $p = b.proc$. Having $a \xrightarrow{rb} b$ implies that on process p there are none or some operations executed after a returns and before b starts. We build T by collecting all of those successive operations, starting with h_1 such that $a \xrightarrow{p} h_1$, and then $h_2, ..., h_q$ with $a \xrightarrow{p} h_1 \xrightarrow{p} h_2 \xrightarrow{p} ... \xrightarrow{p} h_q \xrightarrow{p} b$. We set $T = (h_1, h_2, ..., h_q)$.

$\Leftarrow$)

Let $a, h_1, ..., h_q, b \in H$ with $a \xrightarrow{p} h_1 \xrightarrow{p} ... \xrightarrow{p} h_q \xrightarrow{p} b$ and $p := b.proc$. By definition of $\xrightarrow{p}$, we have for all $1 \leq i \leq q$, $h_i.rtime < h_{i+1}.stime$ as well as $a.rtime < h_1.stime$ and $h_q.rtime < b.stime$. We recall that for all $1 \leq i \leq q$, $h_i.stime < h_i.rtime$. By the transitivity of $<$, we have $a.rtime < b.stime$ which means $a \xrightarrow{rb} b$ by definition.

□

Proposition 4.3. Let $G = (H, \rightarrow) \in \mathcal{G}^m_{Alg1}, m \in \mathbb{N}$. For $a, b \in H$,

$$a \xrightarrow{rb} b \Leftrightarrow a \twoheadrightarrow b.$$

Proof. $\Rightarrow$)

Let $G = (H, \rightarrow) \in \mathcal{G}^m_{Alg1}$, and $a, b \in H$ such that $a \xrightarrow{rb} b$. By Proposition **??**, there exists a tuple $T = (h_1, ..., h_q)$, $h_i \in H, q \in \mathbb{N}$ such that

$$a \xrightarrow{p} h \xrightarrow{p} h_2 \xrightarrow{p} ... \xrightarrow{p} h_q \xrightarrow{p} b$$

with $p = b.proc$.

Lines 7 and 8 of Algorithm **??** ensure that a path always exists from any operation to its direct successor on a given process, if such a successor exists. And since $h_i \xrightarrow{p} h_{i+1}$ is defined to mean that $h_{i+1} \in \sigma(h_i)$, we have

$$a \twoheadrightarrow h_1 \twoheadrightarrow h_2 \twoheadrightarrow ... \twoheadrightarrow h_{q-1} \twoheadrightarrow h_q \twoheadrightarrow b.$$

Thus, $a \twoheadrightarrow b$ by transitivity of $\twoheadrightarrow$.

$\Leftarrow$)

We have $a \twoheadrightarrow b$. Let the $h_1, ..., h_q$ be the vertices on the path from a to b such that

$$a \rightarrow h_1 \rightarrow h_2 \rightarrow ... \rightarrow h_{q-1} \rightarrow h_q \rightarrow b$$

By construction, $\forall c, d \in H : c \rightarrow d \Rightarrow c \xrightarrow{d.proc} d$. Thus, for $p = b.proc$,

$$a \rightarrow h_1 \rightarrow h_2 \rightarrow ... \rightarrow h_{q-1} \rightarrow h_q \rightarrow b \quad \Rightarrow \quad a \xrightarrow{p} h_1 \xrightarrow{p} ... \xrightarrow{p} h_q \xrightarrow{p} b$$

$\Rightarrow a \xrightarrow{rb} b$ by Proposition **??**. □

Remark 4.4. Let $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ a history and $ord(H) = (a_1, ..., a_n)$. Algorithm **??** traverses $ord(H)$ in reverse order; that is, it starts with a_n, then proceeds with a_{n-1}, continuing similarly until a_1. Each operation is processed using lines from 5 to 12 of the algorithm. Suppose the current operation being processed is a_i. Then, upon completion of line 12, a path exists from a_i to all $b \in H$ such that $a \xrightarrow{rb} b$. Consequently, for all operations $a_i, a_{i+1}, \ldots, a_n$, there exists a path to any operation being launched after their completion.

We have established that for a history $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$, each vertex of $G = Alg1(H)$ has at most $m = |\mathbb{P}|$ outgoing edges. We now turn to the complementary property: showing that each vertex has at most m incoming edges.

Lemma 4.5. Let $G = (H, \rightarrow) \in \mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$. Then, for all $a \in H$, $deg_{in}(a) \leq m$.

Proof. We proceed by showing that each vertex has at most one incoming edge from each process. Formally, we aim to prove that for all $c \in H$, no $a, b \in H$ exist such that $a \xrightarrow{ss} b \wedge a \rightarrow c \wedge b \rightarrow c$.

We proceed by contradiction. Suppose that there exists $a, b \in H$ such that $a \rightarrow c \wedge b \rightarrow c \wedge a \xrightarrow{ss} b$.

As a and b are executed on the same process, we either have $a \xrightarrow{rb} b$ or $b \xrightarrow{rb} a$. Let $a \xrightarrow{rb} b$ without loss of generality. We also have $a \xrightarrow{rb} c$ and $b \xrightarrow{rb} c$ as c starts after both a and b finish by definition of $\rightarrow$.

Our algorithm traverses $ord(H)$ in reverse order; meaning that our operations will be treated in the following order: first c, then b and finally a. Between each, other operations may be processed, though not impacting our reasoning, so we will ignore them.

Let p be the process on which a and b are executed.

Here is how our algorithm will proceed:

* Processing operation c: adding the needed out-going edges (will not matter in our proof as those edges are towards operations that occured after c finished, and we are focused of a and b that occur before c)
* Processing operation b: for the moment there are no out-going edges from b. We supposed the existence of the edge $b \rightarrow c$ which we therefore add to the graph, and which implies $b \xrightarrow{rb} c$
* Processing operation a: as we supposed $a \rightarrow c$ to be at some point an edge of the graph we have by construction that c is a direct successor of a. Let $f \in H$ be the operation such that $a \xrightarrow{p} f$; then either $f \xrightarrow{rb} b$ or $f = b$. In either case, the proof holds. Upon the completion of the processing of operation a, we have $a \twoheadrightarrow f$ as well as $f \twoheadrightarrow b$ by Remark **??**. Since $b \rightarrow c$ implies $b \twoheadrightarrow c$, by transitivity of $\twoheadrightarrow$ we have $a \twoheadrightarrow c$. Since there already exists a path from a to c, the edge $a \rightarrow c$ will not be added during the processing of a.
  Hence the contradiction.

We have thus proven that each vertex has at most one in-coming edge from each process. Consequently, across all processes, each vertex can have at most m in-coming edges. □

4.2. Main proof

To prove cutwidth boundedness, we need to define two specific sets of nodes based on the definition of *cut*. These sets are as follows:

* the set of the last operations launched on each process before the cut ℓ, denoted by $\Gamma_\ell(G)$, and

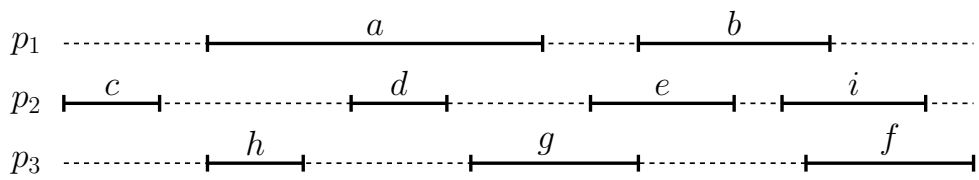


Figure 3: *A history of operations on three processes*

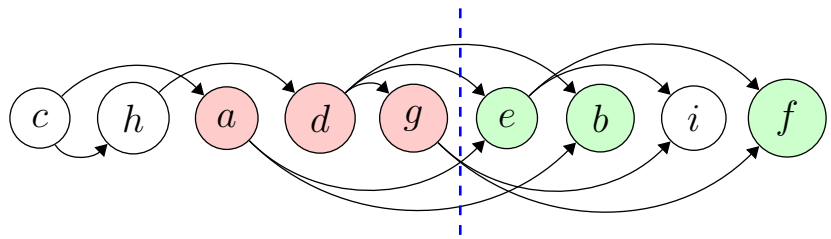


Figure 4: *The graph of the history of* **??**

* the set of the first operations launched on each process after the cut ℓ, denoted by $\Lambda_\ell(G)$

In order to formally define $\Gamma_\ell(G)$ and $\Lambda_\ell(G)$, we first need to introduce the sets $\gamma_p(L)$ and $\lambda_p(L)$:

* the singleton $\gamma_p(L)$ contains the last operation of L to be launched on process p
* the singleton $\lambda_p(R)$ contains the first operation of R to be launched on process p

Definition 4.3. Let $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ be a history with $ord(H) = (a_1, ..., a_n)$, and let $G = Alg1(H)$. For $(L, R) := Cut_\ell(ord, G)$ with $1 \leq \ell \leq n$, and for $p \in \mathbb{P}$, we define

$$\gamma_p(L) := \{a\} \subset L \quad \text{such that} \quad a.proc = p \ \wedge \ (\forall b \in L.\ b \xrightarrow{ss} a \Rightarrow b.stime < a.stime)$$

$$\lambda_p(R) := \{c\} \subset R \quad \text{such that} \quad c.proc = p \ \wedge \ (\forall d \in R.\ d \xrightarrow{ss} c \Rightarrow c.stime < d.stime)$$

We note that it is possible to have $\gamma_p(L) = \emptyset$ and/or $\lambda_p(R) = \emptyset$.

The sets $\Gamma_\ell(G)$ and $\Lambda_\ell(G)$ are now defined as

$$\Gamma_\ell(G) := \bigcup_{p \in \mathbb{P}} \gamma_p(L) \quad \text{and} \quad \Lambda_\ell(G) := \bigcup_{p \in \mathbb{P}} \lambda_p(R) \quad \text{with } (L, R) := Cut_\ell(ord, G)$$

By definition $|\Gamma_\ell(G)|,\ |\Lambda_\ell(G)| \leq |\mathbb{P}|$.
We now introduce notation for the sets of remaining nodes not included in Γ_ℓ or Λ_ℓ.

$$L_\ell := L \setminus \Gamma_\ell \quad \text{and} \quad R_\ell := R \setminus \Lambda_\ell.$$

?? depicts a history and **??** its associated graph. On the graph the red nodes represent the Γ_ℓ set, and the green nodes the Λ_ℓ set of the blue dashed cut ℓ.

If G is the graph of Figure **??** with a cut ℓ being represented by the blue dashed line, the sets are the following:

$$\Gamma_\ell(G) \coloneqq \{a, d, g\} \qquad \Lambda_\ell(G) \coloneqq \{e, b, f\}$$
$$L_\ell(G) \coloneqq \{c, h\} \qquad R_\ell(G) \coloneqq \{i\}$$

Lemma 4.6. Let a history $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and a graph $G = (H, \rightarrow) \in \mathit{Alg1}(H)$. For a cut $(L, R) = \mathit{Cut}_\ell(\mathit{ord}, G)$, with $1 \leq \ell \leq |H|$, there is no edge that crosses the cut ℓ from R to L. That is,

$$\nexists\, a, b \in H : \; a \rightarrow b \;\wedge\; a \in R \;\wedge\; b \in L$$

Proof. We proceed by contradiction. Suppose there is such an edge $a \rightarrow b$ with $a \in R$ and $b \in L$. Algorithm **??** is designed so that any edge $c \rightarrow d$ implies $d \in \sigma(c)$, and therefore c must return before d starts. In particular, $a \rightarrow b$ implies $a.\mathit{rtime} < b.\mathit{stime}$.

However, by the definition of the cut $\mathit{Cut}_\ell(\mathit{ord}, G)$, since $a \in R$ and $b \in L$, we also have $b.\mathit{stime} \leq a.\mathit{stime}$. Combining these inequalities, we obtain:

$$a.\mathit{rtime} < b.\mathit{stime} \leq a.\mathit{stime}$$

which is a contradiction, as it violates the property that $a.\mathit{stime} < a.\mathit{rtime}$. Hence, such an edge cannot exist. □

We present the following table which specifies the types of edges that can cross a cut induced by the linear ordering *ord*. Let $G \in \mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$, and let $(L, R) = \mathit{Cut}_\ell(\mathit{ord}, G)$ be a cut of G. By Lemma **??** no edge originating from $R = \Lambda_\ell \cup R_\ell$ can cross the cut. Therefore, edges with source in R are excluded from the table.

$a \in$	$b \in$	(a, b) can cross the cut ℓ
L_ℓ	Λ_ℓ	✓
L_ℓ	R_ℓ	✗
Γ_ℓ	Λ_ℓ	✓
Γ_ℓ	R_ℓ	✓

?? illustrates a graph containing an edge (a, c) in $L_\ell \times \Lambda_\ell$ (using the same coloring convention as before). Examples of edges in $\Gamma_\ell \times \Lambda_\ell$ and $\Gamma_\ell \times R_\ell$ are shown in **??**. The edges $a \rightarrow e$, $a \rightarrow b$, $d \rightarrow e$, $d \rightarrow b$ are in $\Gamma_\ell \times \Lambda_\ell$; while the edge $g \rightarrow i$ is in $\Gamma_\ell \times R_\ell$.

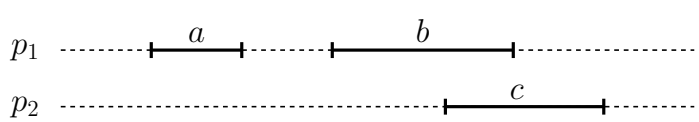


Figure 5: *A history of operations on two processes*

We now aim to prove that no edge in $L_\ell \times R_\ell$ crosses the cut ℓ.

Lemma 4.7. Let a history $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and the graph $G = (H, \rightarrow) \in \mathit{Alg1}(H)$. For a cut $(L, R) = \mathit{Cut}_\ell(\mathit{ord}, G)$, with $1 \leq \ell \leq |H|$, there is no edge that crosses the cut ℓ from $L_\ell(G)$ to $R_\ell(G)$. That is,

$$\nexists\, a, b \in H : \; a \rightarrow b \;\wedge\; a \in L_\ell(G) \;\wedge\; b \in R_\ell(G)$$

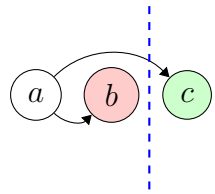


Figure 6: *The graph of the history of* **??**

Proof. We proceed by contradiction. Let suppose the existence of $a \in L_\ell(G), b \in R_\ell(G)$ such that $a \to b$, and let $p = a.proc$ and $q = b.proc$.

As $a \notin \Gamma_\ell(G)$, there exists a vertex $c \in \Gamma_\ell(G)$ executed after a on the same process p . And as $b \notin \Lambda_\ell(G)$, there exists a vertex $d \in \Lambda_\ell(G)$ executed before b on the same process q. By definition of $\Lambda_\ell(G)$ and $\Gamma_\ell(G)$ we have

$$a.rtime < c.stime < d.stime < b.stime.$$

Having $a \to b$ implies that b is a direct successor of a, meaning that b is the first operation to occur on its process q after a finishes. But there exists d on process q which starts after a finishes, and occurs before b. This implies that b is not a direct successor of a. By construction of the algorithm, if b is not a direct successor of a, then the edge $a \to b$ cannot exist. This leads to a contradiction.

Thus, there is no edge in $L_\ell(G) \times R_\ell(G)$ that crosses the cut ℓ. □

Theorem 4.8. Let $G \in \mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$. Then,

$$cutwidth(G) \leq 2m^2$$

Proof. Let a cut $(L, R) = Cut_\ell(ord, G)$, with $1 \leq \ell \leq |H|$. Let $\Gamma_\ell := \Gamma_\ell(G)$, $\Lambda_\ell := \Lambda_\ell(G)$, $L_\ell := L_\ell(G)$, $R_\ell := R_\ell(G)$.

We recall the following results that were established earlier:

* Lemma **??**: for all $a \in H, deg_{out}(a) \leq m$
* Lemma **??**: for all $a \in H, deg_{in}(a) \leq m$
* Lemma **??**: no edge starting from the set R can cross the cut ℓ
* Lemma **??**: there is no edge in $L_\ell \times R_\ell$ that crosses the cut ℓ

Cutwidth is traditionally defined for undirected graphs. However, by Lemma **??**, only edges originating in L can cross the cut at level ℓ. Intuitively, this means that edges cross the cut only "from left to right". For this reason, we retain the directed nature of the graph to maintain clarity throughout the proof. Thus, the core idea of the proof is to bound the number of edges that can "land" on the right-hand side of the cut; that is, the number of edges originating from $L_\ell \cup \Gamma_\ell$ and landing in $\Lambda_\ell \cup R_\ell$

We have $|\Lambda_\ell| \leq m$, and by Lemma **??** each element of Λ_ℓ has at most m incoming edges. Therefore there are at most m^2 edges "landing" in Λ_ℓ.

By Lemma **??**, there are no edges in $L_\ell \times R_\ell$, which implies that each edge crossing the cut ℓ and landing in R_ℓ must originate from Γ_ℓ. Since $|\Gamma_\ell| \leq m$ and Lemma **??** states that each element of Γ_ℓ has at most m outgoing edges, the number of edges in $\Gamma_\ell \times R_\ell$ is also bounded by m^2. This implies that at most m^2 edges cross the cut and land in R_ℓ.

Adding the two bounds, we obtain a total of at most $m^2 + m^2 = 2m^2$ crossing edges, which establishes the result.

□

We now present a corollary that captures the central contribution of this work.

Corollary 4.9. The graph class $\mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$ has uniformly bounded tree-width.

Proof. By Theorem **??**, for all $G \in \mathcal{G}^m_{\mathcal{A}lg1}, m \in \mathbb{N}$, we have $cutwidth(G) \leq 2m^2$. That is to say, the graph class $\mathcal{G}^m_{\mathcal{A}lg1}$ is of uniformly bounded cutwidth. Since by [p. 12-13][**?**] cutwidth is an upper bound over tree-width, it follows that the graph class $\mathcal{G}^m_{\mathcal{A}lg1}$ has uniformly bounded tree-width. □

By modeling our class of history as tree-width bounded graphs, we have shown that any property expressible in MSO logic can be verified in linear time.

5. Consistency verification

We now present an approach for verifying program consistency: modeling histories as finite words. Representing a history as a single word will allow us to use the MONA tool, which takes MSO formulas over words as input and outputs their satisfiability — precisely the computation we desire. The precise syntax of MONA will be detailed in Section **??**. Our exposition proceeds in three steps. First, we formalize consistency models using MSO over histories. Next, we encode histories as words. Finally, we describe the reduction from MSO formulas over histories to equivalent MSO formulas over words, allowing consistency models to be expressed directly as MSO formulas over words. Together, these steps provide the complete foundation for leveraging MONA to verify program consistency.

5.1. MSO logic of histories

5.1.1 Definition

Monadic second order logic (MSO) is the restriction of second-order logic where second-order quantifications are limited to quantifications over sets [**?**]; that is, a logic over objects and sets of objects, which only allows the following quantifications: "for all **objects**","there exists an **object**","for all **sets of objects**" and "there exists a **set of objects**". MSO logic naturally inherits the basic boolean operators **and,or,not**. Sets of objects are denoted by capital letters, whereas individual objects are represented by lowercase letters. Formally, the grammar of the MSO logic of histories is the following, with a and b some operations:

$$
\begin{aligned}
\phi := \; & a.proc = b.proc \\
\mid \; & a.type = b.type \\
\mid \; & a.obj = b.obj \\
\mid \; & a.ival = b.ival \\
\mid \; & a.oval = b.oval \\
\mid \; & t < t \\
\mid \; & \forall a.\phi \\
\mid \; & \exists a.\phi \\
\mid \; & \forall A.\phi \\
\mid \; & \exists A.\phi \\
\mid \; & \phi \wedge \phi \mid \phi \vee \phi \mid \neg\phi \mid (\phi)
\end{aligned}
$$

$$
\begin{aligned}
t := \; & a.stime \\
\mid \; & a.rtime
\end{aligned}
$$

We use the symbol $\equiv$ and notation $a \equiv b$ as "a is logically equivalent to b". The usual implication symbol $\Rightarrow$ is defined in logic as $\phi \Rightarrow \psi \coloneqq \neg\phi \vee \psi$, and the equivalence $\Leftrightarrow$ as

$$\phi \Leftrightarrow \psi \coloneqq (\neg\phi \vee \psi) \wedge (\neg\psi \vee \phi).$$

Also, the symbol $\top$ means "true" and the symbol $\bot$ means "false".

As a quick example, we can translate the following sentence as an MSO formula: "there exists a set of operations of the same type, and one of those operations never returns". In our syntax, an operation does not return if its output value is ∇. This phrase would be written as

$$\exists X.\Big((\forall x.\forall y.x \in X \wedge y \in X \Rightarrow x.type = y.type) \\ \wedge(\exists z.z \in X \wedge z.oval = \nabla)\Big)$$

We can also define predicates, which represent a relation or a property. For instance we can declare a predicate P that takes as input a single variable, and state that $P(a)$ is true if a is the last operation to be launched on its process. In MSO we will rewrite this property as "there is no other operation launched on the same process after a completed". The predicate P is therefore defined as

$$P(a) \coloneqq \neg\exists b.(b.proc = a.proc \wedge b.stime > a.rtime)$$

5.1.2 Formalization of consistency models

Using MSO logic over histories, we can define consistency models as MSO formulas. Our representation of consistency is based on the formulas presented in [**?**]. To formalize consistency properties in monadic second-order logic (MSO), two additional relations over operations are required, as defined in [**?**]: namely, the *visibility* and *arbitration* relations.

" Visibility tells us about the relative timing of update propagation and operations. It is an acyclic relation. If an operation a is visible to b (written $a \xrightarrow{vis} b$), it means that the effect of a is visible to the client performing b. In a system where updates are communicated by messages, this may mean that the message about operation a reached the client that performed operation b before the operation b was performed.

Arbitration is used to indicate how the system resolves update conflicts, *i.e.* how it handles concurrent updates that do not commute. It is a total order on operations. If an operation a is arbitrated before b (written $a \xrightarrow{ar} b$), it means that the system considers the operation a to happen earlier than operation b. In practice, systems can arbitrate operations in various ways; most often, arbitration is represented by some kind of timestamp. The timestamp can be taken from a physical or logical clock on the node that performs the operation, or it can be affixed to the operation later, for example when it is processed on a single server that provides a serialization point."[p. 46][**?**]

"EventualVisibility mandates that, eventually, an operation *op* will be visible to another operation *op′* invoked after the completion of *op*" [p. 10][**?**]. In our framework of finite histories, EventualVisibility consistency cannot be applied, as "eventually" suggests a non finite execution. Thus, in this work, we do not consider the EventualVisibility consistency.

The *return value* consistency model ensures all operations return the appropriate and correct values. In this work, this means that any read operation should output the most recent value written to the

corresponding object. The following formula intuitively captures the following behavior: it holds if all *writes* return the empty value Φ, and if every *read* either accesses an object that has not been written to before, or retrieves the most recent value written to that object.

RVAL :=

$$\forall a. \Big[\Big(a.type = write \Rightarrow a.oval = \Phi\Big) \vee \Big(a.type = read \Rightarrow \Big[\neg\exists x. \Big(x \xrightarrow{ar} a \wedge x.type = write \wedge x.obj = a.obj\Big)$$
$$\vee \exists x. \Big(x.obj = a.obj \wedge x.type = write \wedge x \xrightarrow{ar} a \wedge$$
$$\forall y\ (y \neq x \wedge y.obj = x.obj \wedge y.type = write \Rightarrow y \xrightarrow{ar} x) \wedge x.ival = a.oval\Big)\Big]\Big]$$

Eventual consistency guarantess that all correct operations are eventually applied to all correct replicas, and that all operations return. Moreover, all correct replicas that had the same write operations performed will eventually reach an "equivalent state". An operation or a replica is *correct* if it does not fail. To define eventual consistency, the NOCIRCULARCAUSALITY consistency model must be defined. In [**?**], it is defined as $acyclic(hb)$, where $hb := (\xrightarrow{so} \cup \xrightarrow{vis})^+$ represents the *happens-before* relation, and $\xrightarrow{so}$ denotes the *session order* relation. Specifically, we have $a \xrightarrow{so} b$ if and only if $a \xrightarrow{ss} b \wedge a \xrightarrow{rb} b$. The relation hb is MSO-expressible, as it is a transitive closure. Consequently, $acyclic(hb)$ is also MSO-expressible, since it is a restriction of hb. *Eventual consistency* is defined in [**?**] as the logical conjunction EVENTUALVISIBILITY $\wedge$ NOCIRCULARCAUSALITY $\wedge$ RVAL. However, since EVENTUALVISIBILITY is not relevant in our framework, it is defined as follows.

$$\text{EVENTUALCONSISTENCY} := \text{NOCIRCULARCAUSALITY} \wedge \text{RVAL}$$

Monotonic reads states that "if a process has read a certain value v from an object, any successive read operation will not return any value written before v"[**?**]. Formally, it is defined as follows.

MONOTONICREADS :=

$$\forall a.\forall b.\forall c.\Big(b.type = read \wedge c.type = read \Rightarrow$$
$$((a \xrightarrow{vis} b \wedge b \xrightarrow{so} c) \Rightarrow a \xrightarrow{vis} c)\Big)$$

Read your writes ensures that if a *write* and then a *read* are performed by the same process, the *write* will be visible to the *read*.

READYOURWRITES :=

$$\forall a.\forall b.\Big(a.type = write \wedge b.type = read \Rightarrow (a \xrightarrow{so} b \Rightarrow a \xrightarrow{vis} b)\Big)$$

Real time guarantees that any two operations not concurrently executed are ordered by *arbitration*.

REALTIME :=

$$\forall a.\forall b. a \xrightarrow{rb} b \Rightarrow a \xrightarrow{ar} b$$

Single order defines a global order that defines *visibility* and *arbitration*.

SINGLEORDER :=

$$\exists X.(\forall x. x \in X \Rightarrow x.oval = \nabla) \wedge \forall a.\forall b.(a \xrightarrow{vis} b \Leftrightarrow a \xrightarrow{ar} b \wedge \neg(a \in X))$$

Linearizability, also knows as *strong consistency* states that "each operation shall appear to be applied instantaneously at a certain point in time between its invocation and its response"[**?**]. This "point" is called the linearization point of an operation.

$$\text{LINEARIZABILITY} := \text{SINGLEORDER} \wedge \text{REALTIME} \wedge \text{RVAL}$$

5.2. Encoding histories as words

Words in computer science are strictly similar to the words we read and write everyday. Words are, as we know them, a sequence of letters that belong to a certain alphabet. However there exists several different alphabets: latin alphabet, cyrillic alphabet, greek alphabet, arabic alphabet... Each of those alphabets are sets of symbols. We are free to define a letter as something more than a symbol; a structure that can hold more information than a single symbol. In our case, using vectors as letters is suitable for the information we want a single letter to hold. We will discuss why a vector is appropriate further later; for now we will develop the syntax to be used for words and the semantics behind it.

The usual symbol for representing an alphabet is Σ; hence, we will denote our alphabet by Σ. We also use the $*$ symbol which stands for the Kleene star. The Kleene star is an operation of arity one which for a set S, will be written as S^* and mean that S^* is the set of all sequences of elements of S of length zero or more. For example, if $\Sigma = \{a, b\}$ is an alphabet with letters a and b, Σ^* is the set of all sequences of letters of Σ. We have $\Sigma^* = \{\varepsilon, a, ab, b, ba, abb, aab, aba, bba, abab, ...\}$ and so on. To denote the "empty word" (the word of length zero) we use the symbol ε, and $\varepsilon \in \Sigma^*$. Usually words are described as strings of letters, but as our letters will be vectors and not symbols, we will prefer to use the term "sequence of letters" in the next pages.

Let Σ be our alphabet. Our *letters* will be column vectors of a certain finite dimension n; i.e. each letter will be of the form

$$\begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_{n-1} \\ \alpha_n \end{bmatrix}$$

Each α_i is called a component or a coordinate. When we write $\begin{bmatrix} \vdots \\ \alpha_i \\ \vdots \end{bmatrix}$ we mean that α_i is the i-th component of the vector, i.e. that it is "on the i-th line". Using the symbol # for a component will mean that it can take any value possible given a certain context. For instance if the vectors are defined in $\{0, 1\}^n$ (i.e. are of length n and each component is either 0 or 1), # represents either 0 or 1. The vertical dots $\vdots$ keep their usual meaning, which is for example: $\begin{bmatrix} a \\ \vdots \\ a \end{bmatrix}$ is the vector containing only a, and $\begin{bmatrix} \alpha_1 \\ \vdots \\ \alpha_n \end{bmatrix}$ the vector that has as components $\alpha_1, \alpha_2, \alpha_3$ and so on until α_n.

For an alphabet $\Sigma := \{0, 1\}^2$ we have all the words over Σ defined as $\Sigma^* := (\{0, 1\}^2)^*$, and for instance $\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 1 \end{bmatrix}, \begin{bmatrix} 0 \\ 0 \end{bmatrix} \cdots \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \varepsilon \in \Sigma^*$.

Our goal is to define an encoding function *Enc* that takes as input a history, a visibility relation, and an arbitration relation [**?**], and outputs a word on an alphabet of vectors. We restrict the components of vectors to the value 0 and 1 to fit the requirements of MONA. Thus, for $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$,

$$Enc: \ H \ \mapsto \ Enc(H) \in (\{0, 1\}^{|\mathbb{P}|})^*$$

Each letter of $Enc(H)$ represents the operations and relations of H at a certain point in time. Thus, the word $Enc(H)$ forms a sequence of states of H over time, providing a timeline of its evolution. An important amount of information is needed to be encoded. For a clearer explanation of the Enc function, we will progressively enrich its output, starting with the most basic information and moving towards the most intricate. Examples will be provided along the way. The outline is the following:

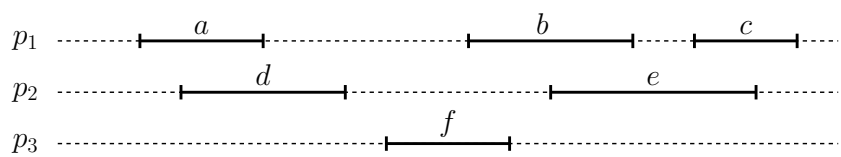
1) Building the timeline
2) Adding information about types and values
3) Representing arbitration and visibility

The most basic information mentioned above is the start and return time of each operation. This information is encoded first.

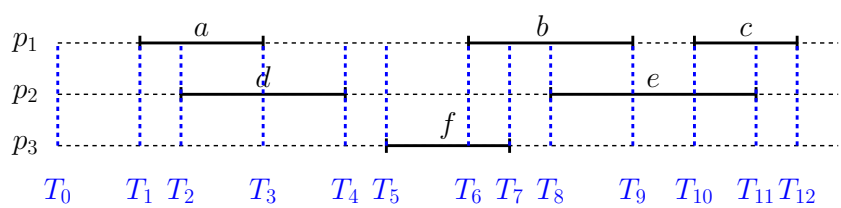
5.2.1 Building the timeline

$Enc(H)$ will represent a timeline we want to read "from left to right", and each of its letters depicts an instant. Our column vector letters will be of length $|\mathbb{P}|$ as $\mathbb{P}$ is finite, and each coordinate will hold information about a single process. When an operation is running on process p, the coordinate of process p will hold 1; and when no operation is running it will hold 0. If a history is run on processes $p_1, p_2, ..., p_i$, the coordinates of a vector of its encoding will hold from top to bottom information about $p_1, p_2, ..., p_i$.

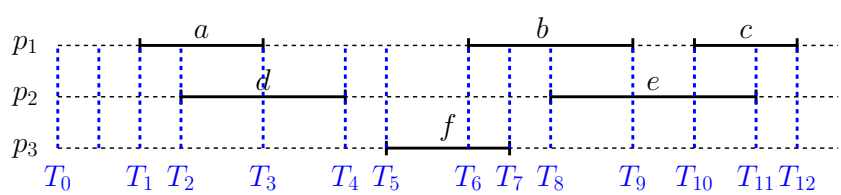
Example 2. The history H_1 below is run on three processes.



Each time an operation starts or returns, we put a mark.



One vector will represent one mark. Let our alphabet be the vectors of length three, and of coordinates 0 or 1, i.e. $\Sigma := \{0,1\}^3$. We have $Enc(H_1) \in (\{0,1\}^3)^*$. The result of its encoding is depicted below, each letter being under the associated mark.



$$\begin{bmatrix}0\\0\\0\end{bmatrix} \begin{bmatrix}1\\0\\0\end{bmatrix} \begin{bmatrix}1\\1\\0\end{bmatrix} \begin{bmatrix}0\\1\\0\end{bmatrix} \begin{bmatrix}0\\0\\0\end{bmatrix} \begin{bmatrix}0\\0\\1\end{bmatrix} \begin{bmatrix}1\\0\\1\end{bmatrix} \begin{bmatrix}1\\0\\0\end{bmatrix} \begin{bmatrix}1\\1\\0\end{bmatrix} \begin{bmatrix}0\\1\\0\end{bmatrix} \begin{bmatrix}1\\1\\0\end{bmatrix} \begin{bmatrix}1\\0\\0\end{bmatrix} \begin{bmatrix}0\\0\\0\end{bmatrix}$$

Each encoding starts with the vector zero (T_0 here). At T_1, operation a starts on process p_1, thereby we set to 1 the bit of its process. When comes T_2, another operation starts while a is still running. We let the p_1 bit as it is, and set the new operation's bit to 1 on p_2's coordinate. On T_3, a that stops running: its bit is set back to 0.

We traverse the history this way until the last operation returns.

Finally,

$$Enc(H_1) = \begin{bmatrix}0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\end{bmatrix}\begin{bmatrix}1\\1\\0\end{bmatrix}\begin{bmatrix}0\\1\\0\end{bmatrix}\begin{bmatrix}0\\0\\0\end{bmatrix}\begin{bmatrix}0\\0\\1\end{bmatrix}\begin{bmatrix}1\\0\\1\end{bmatrix}\begin{bmatrix}1\\0\\0\end{bmatrix}\begin{bmatrix}1\\1\\0\end{bmatrix}\begin{bmatrix}0\\1\\0\end{bmatrix}\begin{bmatrix}1\\1\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\end{bmatrix}\begin{bmatrix}0\\0\\0\end{bmatrix}$$

$\Diamond$

This intuitive explanation of the encoding method is a bit more meticulous to describe as a general algorithm. We first need to order the history on the attributes *stime* and *rtime* (which were marked by the T_i in our example), and then iterate over them. If H is a history, we collect the *stime* and *rtime* data with $Timeline(H)$ that is defined as

$$Timeline(H) \coloneqq$$
$$SortedOnTime\Big(\bigcup_{a\in H}\Big\{(a.proc, a.stime), (a.proc, a.rtime)\Big\} \setminus \{(\cdot, time) \mid time = \Omega\}\Big)$$

where $SortedOnTime$ sorts this set of couples of process—time pairs in ascending order of time; outputting a collection $C = ((p_1, time_1), (p_2, time_2), ...)$. To get the first pair we use $First(C)$, which here is $(p_1, time_1)$; and to get the rest of the collection we use $Tail(C)$, which here is $((p_2, time_2), ...)$.

The notation $a \cdot b$ will be used to mean the concatenation of a and b. If $\begin{bmatrix}\alpha_1 & \dots & \alpha_j\end{bmatrix}$ is a row vector, we write the transpose by $^\top$ and

$$\begin{bmatrix}\alpha_1, \cdots, \alpha_j\end{bmatrix}^\top \coloneqq \begin{bmatrix}\alpha_1\\ \vdots \\ \alpha_j\end{bmatrix} \text{ and } \begin{bmatrix}\alpha_1\\ \vdots \\ \alpha_j\end{bmatrix}^\top \coloneqq \begin{bmatrix}\alpha_1, \cdots, \alpha_j\end{bmatrix}.$$

We will use the transposes of row vectors to represent column vectors in our algorithms, as it facilitates the reading.

Algorithm 2 Basic Timeline

$H_{\mathbb{P}}$ a history over $\mathbb{P} = \{p_1, p_2, ...\}$ finite with $|\mathbb{P}| = n$ $Enc \in (\{0,1\}^n)^*$ a word over the alphabet $\{0,1\}^n$ $C \leftarrow Timeline(H)$ $Enc \leftarrow \begin{bmatrix}0, \cdots, 0\end{bmatrix}^\top \in \{0,1\}^n$ $LastVec \leftarrow \begin{bmatrix}0, \cdots, 0\end{bmatrix}^\top \in \{0,1\}^n$ $C \neq ()$ $(p_i, time) \leftarrow First(C)$ $C \leftarrow Tail(C)$ $\begin{bmatrix}\alpha_1, \cdots, \alpha_n\end{bmatrix}^\top \leftarrow LastVec$ $LastVec \leftarrow \begin{bmatrix}\alpha_1, \cdots, \neg\alpha_i, \cdots, \alpha_n\end{bmatrix}^\top$ $Enc \leftarrow Enc \cdot LastVec$

As an example, we apply this last algorithm to the history shown in Figure **??**, resulting in the encoding of Figure **??**. The full computation is provided in Appendix, Example **??**.

We may now desire to add data about types, values and objects.

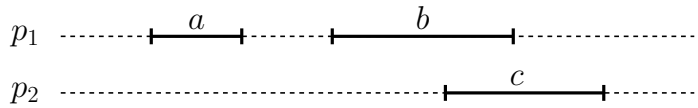


Figure 7: *A history on two processes*

	a	b	c
proc	p_1	p_1	p_2
stime	T_1	T_3	T_4
rtime	T_2	T_5	T_6
type	*write*	*read*	*write*
obj	y	x	x
ival	14	Φ	"*fly*"
oval	Φ	"*bee*"	Φ

Figure 8: *Information for history of Figure* **??**

5.2.2 Adding information about types, values, and objects

For now the only data encoded were the start and return times. We want to enrich the encoding with the *type*, *ival*/*oval* and *obj* attributes. We will do so by adding coordinates to our existing vectors given by the algorithm above. However, we must restrict the set of possible $Types$, $Values$ and $Objects$ to finite sets. Here our $Types$ are only *read* and *write*, but the possible $Values$ are not specified. Therefore we will assume the set $Values$ to be finite. Since we aim to encode such types, values and objects on binary coordinates, we need to encode them a special way. For the read and write operations we can simply state that a *read* is a 0, and a *write* is a 1. For $Values$, as it is a finite and thereby enumerable set, we can assign to each value a number, and convert this number to its binary encoding. For example, if $Values = \{a, b, c\}$ we can assign 0 to a, 1 to b, 2 to c; and encode a as $00_2 = 0_{10}$, b as $01_2 = 1_{10}$ and c as $10_2 = 2_{10}$ (the notation α_β stands for "α is represented in base β"). To encode the values as what we will call "binary words", $\lceil \log_2 |Values| \rceil$ bits are needed.

To encode $Objects$ we use exactly the same method as $Values$; that is to represent objects by binary words (which is possible because $Objects$ is a finite enumerable set).

In Algorithm **??**, two successive vectors differ in only one coordinate. This means, in particular, that when introducing a new operation a on a vector w, we only need to encode data about a. Thus, w is a proper letter to hold more information about a, such as its *type*,*ival*/*oval* and *obj*.

Since $\lceil \log_2 |Types| \rceil = \lceil \log_2(2) \rceil = 1$, each vector will hold $|\mathbb{P}| + 1 + \lceil \log_2 |Values| \rceil + \lceil \log_2 |Objects| \rceil$ coordinates.

For $\mathbb{P} = \{p_1, ...p_n\}$, and H a history, each letter of $Enc(H)$ will be of the form

$$\begin{bmatrix}0\\0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}\begin{bmatrix}1\\1\end{bmatrix}\begin{bmatrix}0\\1\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix}$$

Figure 9: *Encoding of history of Figure* **??** *with Algorithm* **??**

$$\begin{bmatrix} p_1 \\ \vdots \\ p_n \\ t \\ v_1 \\ \vdots \\ v_k \\ o_1 \\ \vdots \\ o_\ell \end{bmatrix}$$

} time attribute, 0 for *read* and 1 for *write*

} binary encoding of the *ival* and *oval* if $t = 0$

A *read* operation does not have an input value and a *write* operation does not have an output value, therefore we either encode the *ival* or *oval* attribute as only one is relevant at a time. With the encoding of the *type* we deduce whether $v_1 \dots v_k$ encodes an *ival* or *oval* attribute.

To formally present this encoding, a bijective function to transform a value into a binary word is necessary. We define it as

$$f \ : Values \ \mapsto \ \{0,1\}^k \quad \text{with } k = \lceil \log_2 |Values| \rceil$$

For the example provided before with $Values = \{a, b, c\}$ and 0 assigned to a, 1 to b and 2 to c; we have $f(a) = 00$, $f(b) = 01$ and $f(c) = 10$.

The bijective function to convert an object to a binary word is

$$g \ : Objects \ \mapsto \ \{0,1\}^\ell \quad \text{with } \ell = \lceil \log_2 |Objects| \rceil$$

We also define a function *ioput* that takes as input an operation and outputs its *ival* if it is a *write* and outputs its *oval* if it is a *read*:

$$\begin{aligned} ioput \ &: H \ \rightarrow \ Values \\ h \ &\mapsto \begin{cases} ival & \text{if } h.type = write \\ oval & \text{if } h.type = read \end{cases} \end{aligned}$$

We enrich the $Timeline$ function's output with the needed data and call this new function $Timeline_2$:

$$Timeline_2(H) \coloneqq$$
$$SortedOnTime_2\Big(\bigcup_{a \in H} \Big\{ (a.proc, a.stime, a.type, ioput(a), a.obj), (a.proc, a.rtime, a.type, ioput(a), a.obj) \Big\}$$
$$\setminus \{(\cdot, time, \cdot, \cdot, \cdot) \mid time = \Omega\} \Big)$$

where $SortedOnTime_2$ sorts this set of process-time-type-value tuples in ascending order of time; outputting a collection $C = ((p_1, time_1, type_1, value_1, obj_1), ...)$.

To get the first tuple we use $First(C)$, which here is $(p_1, time_1, type_1, value_1, obj_1)$; and to get the rest of the collection we use $Tail(C)$, which here is $((p_2, time_2, type_2, value_2, obj_2), ...)$.

Below is Algorithm **??** enriched with the types and values data encoding. For a better readability we highlight in blue the bits of code modified, and in pink the added parts. We comment the code in teal.

$$\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\1\\0\\0\\1\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\\0\\1\\0\end{bmatrix}\begin{bmatrix}1\\1\\1\\1\\0\\0\end{bmatrix}\begin{bmatrix}0\\1\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}$$

Figure 10: *Encoding of history of Figure* **??** *with Algorithm* **??**

Algorithm 3 Basic Timeline+TypesValuesObjects

$H_{\mathbb{P}}$ a history over $\mathbb{P} = \{p_1, ..., p_n\}$ finite with $|\mathbb{P}| = n$, $Types = \{read, write\}$, $k = \lceil \log_2 |Values| \rceil$, $\ell = \lceil \log_2 |Objects| \rceil$ and $m = n + k + \ell + 1$ $Enc \in (\{0,1\}^m)^*$ a word over the alphabet $\{0,1\}^m$ $C \leftarrow Timeline_2(H)$ $Enc \leftarrow \left[0, \cdots, 0\right]^\top \in \{0,1\}^m$ $LastVec \leftarrow \left[0, \cdots, 0\right]^\top \in \{0,1\}^m$ $C \neq ()$ $(p_i, time, type, value, obj) \leftarrow First(C)$ $C \leftarrow Tail(C)$ $\left[\alpha_1, \cdots, \alpha_m\right]^\top \leftarrow LastVec$ $\alpha_i = 0$ [new operation to encode] $\gamma_1 \ldots \gamma_k \leftarrow f(value)$ $\lambda_1 \ldots \lambda_\ell \leftarrow g(obj)$ $type = read$ $t \leftarrow 0$ $t \leftarrow 1$ $LastVec \leftarrow \left[\alpha_1, \cdots, \neg\alpha_i, \cdots, \alpha_n, t, \gamma_1, \cdots, \gamma_k, \lambda_1, \cdots, \lambda_\ell\right]^\top$ $LastVec \leftarrow \left[\alpha_1, \cdots, \neg\alpha_i, \cdots, \alpha_n, 0, \cdots, 0\right]^\top \in \{0,1\}^m$ $Enc \leftarrow Enc \cdot LastVec$

As an example, we apply this last algorithm to the history shown in Figure **??**, resulting in the encoding of Figure **??**. The full computation is provided in Appendix, Example **??**.

Finally, in [**?**] are described two other important relations: arbitration and visibility. We cite:

" Visibility tells us about the relative timing of update propagation and operations. Is is an acyclic relation. If an operation a is visible to b (written $a \xrightarrow{vis} b$), it means that the effect of a is visible to the client performing b. In a system where updates are communicated by messages, this may mean that the message about operation a reached the client that performed b before the operation b was performed. " [p. 46][**?**]

" Arbitration is used to indicate how the system resolves *update conflicts, i.e.* how it handles concurrent updates that do not commute. It is a total order on operations. If an operation a is arbitrated before b (written $a \xrightarrow{ar} b$), it means that the system considers the operations a to happen earlier than operation b. " [p. 46][**?**]

The next step is to encode these two relations.

5.2.3 Representing arbitration and visibility

To represent arbitration and visibility, some constraints are needed. We will assume the RealTime [**?**] consistency model which states that if an operation a returns before operation b starts, then a is arbitrated before b. This assumption will significantly simplify our encoding technique while still remaining realistic. Indeed, if an operation ends before another one starts, it is natural to assume that this first operation has been "done" before the other. With this restriction, the only arbitration to encode is between two operations being run concurrently at a given time. The "given time" will mean,

here, on a single vector of our encoding. That is, if two operations a and b are running concurrently on two different processes, which one is arbitrated before the other ? a before b or b before a?

Arbitration at a given time t, can be represented by a matrix where rows and columns are indexed by the processes. For the set of processes $\mathbb{P} = \{p_1, \dots, p_n\}$ the matrix will be of the form

$$\begin{array}{c c} & \begin{array}{cccc} p_1 & & & p_n \end{array} \\ \begin{array}{c} p_1 \\ \\ \\ \\ p_n \end{array} & \begin{bmatrix} 0 & \alpha_{12} & & \alpha_{1n} \\ \neg\alpha_{12} & & & \\ & & & \alpha_{(n-1)n} \\ \neg\alpha_{1n} & & \neg\alpha_{(n-1)n} & 0 \end{bmatrix} \end{array} \in \mathcal{M}_{n,n}(\{0,1\})$$

where $\mathcal{M}_{n,n}(\{0,1\})$ denotes the set of $n \times n$ matrices with entries in $\{0,1\}$. A variable α_{ij} is set to 1 if the operation on process i is arbitrated before the operation on process j, and is set to 0 if not. Since here we either have i arbitrated before j (shortened $i \xrightarrow{ar} j$) or j arbitrated before i (shortened $j \xrightarrow{ar} i$), if we have $i \xrightarrow{ar} j$ it means that we have $\neg(j \xrightarrow{ar} i)$; which is the reason we use $\neg$ is the matrix. This primarily means that for processes $\{p_1, \dots, p_n\}$ the only α_{ij} needed to encode arbitration are the one of the upper triangular section of the matrix above; that is a total of $(n-1) + (n-2) + ... + 1 = \frac{(n-1)n}{2}$ values. Indeed the first row contains $n-1$ relevant values, the second $n-2$, and so on until the second to last row of 1 value only. The diagonal is not relevant, as an operation cannot be arbitrated before itself; therefore all diagonal values are automatically set to zero.

On three processes p_1, p_2 and p_3, the matrix will look like

$$\begin{array}{c c} & \begin{array}{ccc} p_1 & p_2 & p_3 \end{array} \\ \begin{array}{c} p_1 \\ p_2 \\ p_3 \end{array} & \begin{bmatrix} 0 & \alpha_{12} & \alpha_{13} \\ \neg\alpha_{12} & 0 & \alpha_{23} \\ \neg\alpha_{13} & \neg\alpha_{23} & 0 \end{bmatrix} \end{array} \in \mathcal{M}_{3,3}(\{0,1\})$$

and the relevant values are α_{12}, α_{13} and α_{23}; that respectively represent if $p_1 \xrightarrow{ar} p_2, p_1 \xrightarrow{ar} p_3$ and $p_2 \xrightarrow{ar} p_3$. In our encoding we can store those α_{ij} in increasing order of i and then j; so here it will be α_{12}, α_{13} and then α_{23}. If we had to have a history over those three processes, a vector containing the arbitration relation data would be of the form

$$\begin{bmatrix} p_1 \\ p_2 \\ p_3 \\ t \\ v_1 \\ \vdots \\ v_k \\ o_1 \\ \vdots \\ o_\ell \\ \alpha_{12} \\ \alpha_{13} \\ \alpha_{23} \end{bmatrix}$$

} *[illegible]* ??

Continuing with our example of Figure **??**: it is a history on 2 processes, so a single α_1 can encode arbitration:

$\alpha_1 = \top \Leftrightarrow p_1 \xrightarrow{ar} p_2, \quad \alpha_1 = \bot \Leftrightarrow p_2 \xrightarrow{ar} p_1.$

The encoding we had so far of this history is:

$$\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\1\\0\\0\\1\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\\0\\1\\0\end{bmatrix}\begin{bmatrix}1\\1\\1\\1\\0\\0\end{bmatrix}\begin{bmatrix}0\\1\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}$$

Let $b \xrightarrow{ar} c$. We add a single coordinate to our vectors to be able to encode arbitration:

$$\begin{array}{ccccccc}
\begin{bmatrix}0\\0\\0\\0\\0\\0\\\#\end{bmatrix} & \begin{bmatrix}1\\0\\1\\0\\0\\1\\\#\end{bmatrix} & \begin{bmatrix}0\\0\\0\\0\\0\\0\\\#\end{bmatrix} & \begin{bmatrix}1\\0\\0\\0\\1\\0\\\#\end{bmatrix} & \begin{bmatrix}1\\1\\1\\1\\0\\0\\\#\end{bmatrix} & \begin{bmatrix}0\\1\\0\\0\\0\\0\\\#\end{bmatrix} & \begin{bmatrix}0\\0\\0\\0\\0\\0\\\#\end{bmatrix} \begin{array}{l}p_1\\p_2\\\\\\\\\\p_1 \xrightarrow{ar} p_2\ ?\end{array}\\
T_1 & T_2 & T_3 & T_4 & T_5 & T_6 & T_7
\end{array}$$

We recall that **for any given time T_i there is a matrix** containing data about arbitration. This matrix is relevant **if and only if** at least two operations are concurrently being executed at this moment, that is there are at least two bits set to 1 among the processes' coordinates. In our example this occurs only at T_5, where b and c are run in parallel. Since $b \xrightarrow{ar} c$ and $b.proc = p_1, c.proc = p_2$, we put T_5's bit to 1 and all the others T_i's bit to zero, as they are not relevant. The encoding is now the following:

$$\begin{bmatrix}0\\0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\1\\0\\0\\1\\0\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\\0\\1\\0\\0\end{bmatrix}\begin{bmatrix}1\\1\\1\\1\\0\\0\\1\end{bmatrix}\begin{bmatrix}0\\1\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\\0\end{bmatrix}$$

This approach requires to keep track of which operations are being currently run. It can be tackled by defining a set named $Current$ that contains current operations. Thereby $Current \subset H$. The reviewed timeline function $Timeline_3$ is defined as

$$Timeline_3(H) :=$$
$$SortedOnTime_3\Big(\bigcup_{a \in H} \big\{(a, a.stime), (a, a.rtime)\big\} \setminus \{(\cdot, time) \mid time = \Omega\}\Big)$$

where $SortedOnTime_3$ sorts this set of operation-time couples in ascending order of time; outputting a collection $C = ((op_1, time_1), ...)$.

To get the first tuple we use $First(C)$, which here is $(op_1, time_1)$; and to get the rest of the collection we use $Tail(C)$, which here is $((op_2, time_2), ...)$.

For $|\mathbb{P}| = \{p_1, ..., p_n\}$, arbitration is encoded on $a = \frac{(n-1)n}{2}$ values. For a better readability we highlight in blue the bits of code modified to encode arbitration, and in pink the added parts. We comment the code in teal. In our algorithm we have that

* $Current$ is a list indexed from 1 to $length(Current)$ included
* list concatenation is denoted as $\cdot$
* and $Current.remove(op)$ removes op from $Current$ (suitable here, as operations are unique in the list)

Algorithm 4 Basic Timeline+TypesValuesObjects+Ar

$H_{\mathbb{P}}$ a history over $\mathbb{P} = \{p_1, ..., p_n\}$ finite with $|\mathbb{P}| = n$, $Types = \{read, write\}$, $k = \lceil \log_2 |Values| \rceil$, $\ell = \lceil \log_2 |Objects| \rceil$, $a = \frac{(n-1)n}{2}$ and $m = n + k + \ell + 1 + a$ $Enc \in (\{0,1\}^m)^*$ a word over the alphabet $\{0,1\}^m$ $C \leftarrow Timeline_3(H)$ $Enc \leftarrow \left[0, \cdots, 0\right]^\top \in \{0,1\}^m$ $LastVec \leftarrow \left[0, \cdots, 0\right]^\top \in \{0,1\}^m$ $Current \leftarrow [\ ]$ $C \neq ()$ $(op, time) \leftarrow First(C)$ $C \leftarrow Tail(C)$ $\left[\alpha_1, \cdots, \alpha_m\right]^\top \leftarrow LastVec$ $\alpha_i = 0$ [new operation to encode] $\gamma_1 \ldots \gamma_k \leftarrow f(ioput(op))$ $\lambda_1 \ldots \lambda_\ell \leftarrow g(op.obj)$ $type = read$ $t \leftarrow 0$ $t \leftarrow 1$ $Current \leftarrow Current \cdot [op]$ $L \leftarrow length(Current)$ $\delta_1 \ldots \delta_a \leftarrow \left[0, \cdots, 0\right]^\top \in \{0,1\}^a$ $c \leftarrow 1$ $L \geq 2$ $1 \leq j < L$ $j < q \leq L$ with $j, q \in \mathbb{N}$ $Current[j] \xrightarrow{ar} Current[q]$ $\delta_c = 1$ $c \leftarrow c + 1$ $LastVec \leftarrow \left[\alpha_1, \cdots, \neg\alpha_i, \cdots, \alpha_n, t, \gamma_1, \cdots, \gamma_k, \lambda_1, \cdots, \lambda_\ell, \delta_1, \cdots, \delta_a\right]^\top$ $Current \leftarrow Current.remove(op)$ $LastVec \leftarrow \left[\alpha_1, \cdots, \neg\alpha_i, \cdots, \alpha_n, 0, \cdots, 0\right]^\top \in \{0,1\}^m$ $Enc \leftarrow Enc \cdot LastVec$

Visibility is challenging to encode due to its message-passing nature. Intuitively, visibility captures which information has been propagated and is therefore "visible" to certain operations. We recall that $a \xrightarrow{vis} b$ denotes that it "may mean that the message about operation a reached the client that performed operation b before operation b was performed"[**?**]. According to this definition, $a \xrightarrow{vis} b$ can hold only if a starts before b ends. Indeed, if $b \xrightarrow{rb} a$, then b could not have observed the effects of a, since a is starting after the completion of b. Thus, a way to represent visibility with respect to an operation a, is to encode, for each process p, the most recent operation initiated on p that is visible to a.

As a modeling convenience, we assumed for arbitration that if $a \xrightarrow{rb} b$, then $a \xrightarrow{ar} b$. Similarily for visibility we assume that if $a \xrightarrow{ss} b$ and $a \xrightarrow{rb} b$, then $a \xrightarrow{vis} b$. Moreover, we assume that for each pair of processes (p,q), updates from p to q are propagated in a FIFO manner. That is, if two operations a and b are executed on p with $a \xrightarrow{rb} b$, and an operation c is executed on q, then if c has not received the update from a, it cannot receive the update from b. Formally $\neg(a \xrightarrow{vis} c) \Rightarrow \neg(b \xrightarrow{vis} c)$.

For $a \in H$, with $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$ and $p \in \mathbb{P}$, we define:

$$\xi_p(a) := ord(\ \{b \mid b \in H \wedge b.proc = p \wedge b.stime < a.rtime\}\ ) = (c_n, \cdots, c_2, c_1)$$

the ordered tuple of operations starting on process p before a end.

Using this notation, we aim to encode, for each $p \in \mathbb{P}$, the integer i, with $i \in \{1, \ldots, n\}$, such that $c_i \xrightarrow{vis} a$ and $\neg(c_{i-1} \xrightarrow{vis} a)$. We denote this index by $\mu_p(a)$. This index is uniquely defined under the assumption that $c_n \xrightarrow{vis} \cdots \xrightarrow{vis} c_2 \xrightarrow{vis} c_1$, and due to the FIFO propagation property, which ensures that $\neg(c_i \xrightarrow{vis} a)$ implies $\neg(c_{i-1} \xrightarrow{vis} a)$.

Since the encoding operates on finite vectors, we must set an upper bound on this index. If all such indices are bounded by 2^S for some $S \in \mathbb{N}$, then up to S bits per process are required for the encoding. As previously, this information is encoded on the first bit vector that indicates the start of an operation a. For $\mathbb{P} = \{p_1, \cdots, p_n\}$, the bit vectors are now of the following form.

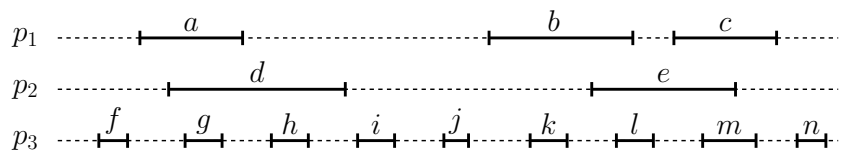


Figure 11: *A history of operations on three processes*

	a	b	c	d	e	f	g	h	i	j	k	l	m	n
p_1	$\varnothing$	1	1	1	2	$\varnothing$	1	1	1	1	2	1	2	1
p_2	$\varnothing$	1	1	$\varnothing$	1	$\varnothing$	1	1	1	1	1	1	1	1
p_3	3	4	2	3	5	$\varnothing$	1	1	1	1	1	1	1	1

op . p

Figure 12: *Visibility table of history of Figure* **??** *containing the* $\mu_p(op)$ *values.*

$$\begin{bmatrix} \# \\ \vdots \\ \# \\ p_{1,1} \\ \vdots \\ p_{1,S} \\ p_{2,1} \\ \vdots \\ p_{2,S} \\ \\ p_{n,1} \\ \vdots \\ p_{n,S} \end{bmatrix}$$

. } previous encoding

} binary representation of $\mu_{p_2}(a)$

We consider, as an example, the history depicted in Figure **??** with visibility relation defined in Figure **??**. The symbol $\varnothing$ denotes the absence of visibility. For instance, $\mu_{p_1}(f) = \varnothing$ because f has no operation visible on process p_1. The $\varnothing$ symbol will be encoded as a 0. In the Figure **??**, the greatest index value is 5; thus, encoding visibility requires $\lceil \log_2 5 \rceil = \lceil 2.3219 \rceil = 3$ bits per process. The encoding corresponding to Figure **??** is shown below. We omit the details of the encoding for types, values, objects, and arbitration.

$$
\begin{array}{ccccccc

5.3. MSO logic of words of bit vectors

We express consistency models in MSO logic over bit vectors that encode histories. To enable automated verification with MONA, we restrict this logic to the WS1S fragment supported by MONA. WS1S — Weak Second-Order Logic of One Successor — allows quantification over integers and sets of integers. These integers are referred to as positions, for the following reason. In MONA, sets $P = \{1, 4\}$ and $Q = \{1, 2, 3\}$ would be represented as

$$\begin{array}{c} P \\ Q \\ \\ \end{array}\begin{array}{ccccc} \begin{bmatrix}0\\0\end{bmatrix} & \begin{bmatrix}1\\1\end{bmatrix} & \begin{bmatrix}0\\1\end{bmatrix} & \begin{bmatrix}0\\1\end{bmatrix} & \begin{bmatrix}1\\0\end{bmatrix} \\ 0 & 1 & 2 & 3 & 4 \end{array} \Big\} \text{natural numbers}$$

with bits set to 1 indicating that the corresponding indexed integer belongs to the set. Hence the use of the term "position". The MONA User Manual [**?**] proposes the following example: a program that verifies the satisfiability of P and Q being sets of positions with $P \setminus Q = \{0, 4\} \cup \{1, 2\}$. The MONA code is the following [p. 4][**?**].

```
var2 P,Q;
P\Q = {0,4} union {1,2};
```

Running the code above, saved in a file named `example.mona`, with the command `mona example.mona` outputs several lines. The relevant outputs here are the counter-example and the satisfying example.

```
ANALYSIS
A counter-example of least length (0) is:
P               X
Q               X

P = {}
Q = {}

A satisfying example of least length (5) is:
P               X 11101
Q               X 000X0

P = {0,1,2,4}
Q = {}
```

5.3.1 Definition

We define the MSO logic of words over bit vectors as follows, restricted to WS1S logic. Lowercase letters represent positions and uppercase letters represent sets of positions.

$$\begin{aligned}
\psi := \; & \forall x.\psi \\
\mid \; & \exists x.\psi \\
\mid \; & \forall X.\psi \\
\mid \; & \exists X.\psi \\
\mid \; & x = \text{succ}(y) \quad \text{with } \text{succ}(y) := y + 1 \\
\mid \; & x = y \quad \text{equality of positions} \\
\mid \; & x \in X \quad \text{“position } x \text{ is in the set } X\text{”} \\
\mid \; & \psi \wedge \psi \mid \psi \vee \psi \mid \neg\psi \mid (\psi)
\end{aligned}$$

Using the formula $x \notin X \equiv \neg(x \in X)$, the statement "there exists P and Q being sets of positions with $P \setminus Q = \{0,4\} \cup \{1,2\}$" is expressed with this logic as

$$\exists P.\ \exists Q.\ \forall x.\ \Big((x \in P \wedge x \notin Q) \Leftrightarrow (x = 0 \vee x = 1 \vee x = 2 \vee x = 4)\Big)$$

We may now "translate" the grammar of the MSO logic of histories into the grammar of the MSO logic of words of bit vectors encoding histories.

5.3.2 Translation from MSO of histories to MSO of words of bit vectors

We present the translation for quantification. We recall that for formulas in MSO logic of histories, a is an operation and A a set of operations; and that for MSO logic of words of bit vectors, x is an integer and X a set of integers. Let $P_i, i \in \{1, ..., n\}$ be the sets representing the rows encoding the processes. We define a predicate $op(x)$ in the MSO of words over bit vectors, which holds if x is a position corresponding to an operation.

$$\begin{aligned} op(x) := \Big(& (x \in P_1 \wedge \exists y.\mathrm{succ}(y) = x \wedge y \notin P_1) \vee \\ & \vdots \\ & (x \in P_{n-1} \wedge \exists y.\mathrm{succ}(y) = x \wedge y \notin P_{n-1}) \vee \\ & (x \in P_n \wedge \exists y.\mathrm{succ}(y) = x \wedge y \notin P_n) \Big) \end{aligned}$$

MSO of histories	MSO of words of bit vectors
$\forall a.\phi$	$\forall x.op(x) \Rightarrow \psi$
$\exists a.\phi$	$\exists x.op(x) \wedge \psi$
$\forall A.\phi$	$\forall X.\Big((\forall x.x \in X \Rightarrow op(x)) \Rightarrow \psi\Big)$
$\exists A.\phi$	$\exists X.\Big((\forall x.x \in X \Rightarrow op(x)) \wedge \psi\Big)$

As an example, we encoded in MONA the RealTime consistency property for two processes, using slightly different constraints and representations of arbitration and visibility, as explained in the code comments.

Example 3. RealTime consistency property expressed in MONA.

```
var2 Process_P, Process_Q;      # 1 if an operation is currently being run, else 0
var2 Process_P_type,Process_Q_type;     # 0 if  READ, 1 if WRITE
var2 Process_P_value,Process_Q_value;       # input value if WRITE, output value if READ
var2 P_arbitrated_before_Q;     # 1 if true, else 0
var2 P_visible_to_Q;        #1 if the current operation on Q is visible by
                        the last operation done before on P, 0 if visible by the second to last
var2 Q_visible_to_P;        #1 if the current operation on P is visible by
                        the last operation done before on P, 0 if visible by the second to last
```

```
pred is_operation(var1 a)=
    (a-1 notin Process_P & a in Process_P) | (a-1 notin Process_Q & a in Process_Q);

pred is_end(var1 b,var1 e)=
    (
        b in Process_P & b-1 notin Process_P =>
        (b <= e & (all1 a: (a >=b & a <= e) => a in Process_P))
    ) | (
        b in Process_Q & b-1 notin Process_Q =>
        (b <= e & (all1 a:(a >= b & a <= e) => a in Process_Q))
    );

pred arbitrated_before(var1 a, var1 b)=
    ex1 end_a,end_b:
    (
        (a in Process_P & b in Process_P |
                a in Process_Q & b in Process_Q) & is_end(a,end_a) & end_a<b-1
    )|(
        is_end(a,end_a) & is_end(b,end_b) & (
            ex1 i : i in P_arbitrated_before_Q =>  (i>=a & i<=end_a & i>=b & i<=end_b)
        )
    );

pred rb(var1 a,var1 b)=
    (
        ex1 end_a : is_end(a,end_a) & end_a<b
    );


pred real_time()=
    (all1 a: is_operation(a) => (
        all1 b: is_operation(b) => (rb(a,b) => arbitrated_before(a,b))
    ));

real_time();
```

◊

The complete translation of the MSO grammar of histories into the grammar of MSO over words of bit vectors, as well as the translation of the previously mentioned MSO formulas of histories for consistency models, is still ongoing. This work could not be finalized during the internship due to limited time and may be continued in the future.

6. Conclusion

In this work, we have shown that any MSO-definable property is decidable in linear time over finite execution traces in replicated data systems. Building on this, we introduced an encoding of execution traces suitable for consistency verification using the MONA tool under minimal assumptions about the structure of these traces. This approach enables a general verification framework that is not limited to a specific consistency model. In contrast to recent work focusing on particular models — such as verification techniques for strong eventual consistency [**?**] or weak consistency [**?**] — our framework supports reasoning about a broader class of consistency properties within a unified logical setting. Since our encoding can be used with MONA, which builds automata from logical specifications, this effectively allows us to link execution traces of replicated data systems to automata.

Knowing that a trace can be represented by an automaton is valuable for several reasons. Since our framework is restricted to finite processes and histories, the execution of a program over these processes can be decomposed into a separate automaton per process. Each process is therefore associated with a finite automaton that captures the specific code fragments it executes. To verify that any history resulting from the parallel execution of these automata satisfies a given consistency model, we proceed in two steps: first we define a class of histories that captures all possible outcomes of the program; and then verify whether each of these histories satisfies the specified consistency property. These constraints can be formalized as intersections of finite automata. Since each finite automaton corresponds to a regular language, the problem reduces to checking whether the intersection language is empty. If the intersection is empty, then no execution of the program satisfies the consistency model.

However, our work is currently restricted to finite domains. An immediate extension would be to investigate the decidability of MSO-definable properties over infinite traces. The current decidability result relies on an algorithmic construction, which is only applicable to finite traces. Nonetheless, if the underlying graph structure of execution traces can be expressed purely in logical terms — using unions, intersections, and negations of relations, rather than an explicit algorithm — then it is possible to extend the decidability result to infinite traces.

Overall, our approach offers a new perspective in the field of consistency verification and opens several avenues for future improvements, such as support for infinite executions and even weaker assumptions regarding visibility and arbitration.

Acknowledgments

I would like to warmly thank the i3s laboratory, and especially my internship supervisors, Étienne and Cinzia, for their support and guidance throughout this experience. Having known them for several years since my bachelor studies, I was particularly happy to meet them again and work with them during this internship. I also thank the team members who attended the seminar during which I presented my work, in particular my fellow interns and PhD students — Anna, Anderson, Gabriele, Erik, and Dalim — who welcomed me and made this experience all the more enriching, as well as Françoise and Pascal, with whom I also had valuable discussions on interesting points to explore in the continuation of my work. I would also like to sincerely thank the members of the jury for dedicating their time and attention to reading my work.

7. Appendices

7.1. Tree-width definition

We adopt the definition of tree-width as proposed in [**?**].

Let $G = (V, E)$ be a graph. A tree-decomposition of G is a pair $\Big(\{X_i \mid i \in I\}, T = (I, F)\Big)$, with $\{X_i \mid i \in I\}$ a family of subsets of V, and T a tree, with the following properties:

- $\bigcup_{i \in I} X_i = V$
- For every edge $e = (v, w) \in E$, there is a subset X_i, $i \in I$ with $v \in X_i$ and $w \in X_i$.
- For all $i, j, k \in I$, if j lies on the path in T from i to k, then

$$X_i \cap X_k \subseteq X_j.$$

The *tree-width* of a tree-decomposition $\Big(\{X_i \mid i \in I\}, T\Big)$ is

$$\max_{i \in I} |X_i| - 1.$$

The tree-width of G, denoted by tree-width(G), is the minimum tree-width of a tree-decomposition of G, taken over all possible tree-decompositions of G.

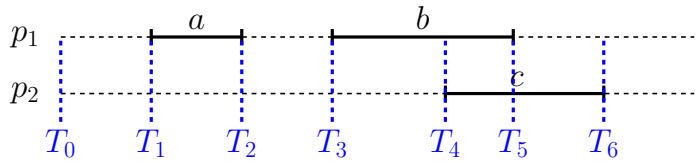
7.2. Summary of Binary Relations and Other Useful Notations

Let $\mathbb{P}$, $\mathbb{T} = \{read, write\}$, $\mathbb{O}$, and $\mathbb{V}$ be finite sets representing, respectively, processes, operation types, objects, and values. Let $H \in \mathcal{H}(\mathbb{P}, \mathbb{T}, \mathbb{O}, \mathbb{V})$, and let $G = (H, \rightarrow) \coloneqq \text{Alg1}(H)$.

NOTATION	*Domain*	*Name*	*Meaning*	*Formally*
$a \xrightarrow{rb} b$	$H \times H$	Returns before	a returns before b starts	$a.rtime < b.stime$
$a \xrightarrow{ss} b$	$H \times H$	Same session	a and b are launched by the same process	$a.proc = b.proc$
$a \overset{p}{\rightharpoonup} b$	$H \times H$ and $p \in \mathbb{P}$	Direct succession on a process	b is the direct successor of a on process p	$b.proc = p \ \wedge \ \neg\exists c.(c \in H \wedge c \neq b \wedge c \xrightarrow{ss} b \wedge a.rtime < c.stime \wedge c.rtime < b.stime\ )$
$\sigma(a)$	H	Direct successors set	All the direct successors of a across all processes	$\bigcup_{p \in \mathbb{P}} \{b \mid b \in H \wedge a \overset{p}{\rightharpoonup} b\}$
$a \rightharpoonup b$	$H \times H$	Direct succession	b is a direct successor of a	$b \in \sigma(a)$
$a \rightarrow b$	$H \times H$	Edge	There is a directed edge from a to b in the graph	
$a \twoheadrightarrow b$	$H \times H$	Path	There exists a directed path from a to b in the graph	$\exists\, h_1, \cdots, h_n \in H$ such that $a \rightarrow h_1 \rightarrow h_2 \rightarrow \ldots \rightarrow h_n \rightarrow b$

7.3. Examples

Example 4. First we display the timestamps at which operations start and/or finish.



We have that a writes 14 to object y, b reads "*bee*" from object x and c writes "fly" to object x. We recall the attributes of those operations:

	a	b	c
$proc$	p_1	p_1	p_2
$stime$	T_1	T_3	T_4
$rtime$	T_2	T_5	T_6
$type$	$write$	$read$	$write$
obj	y	x	x
$ival$	14	Φ	"fly"
$oval$	Φ	"*bee*"	Φ

From this table we extract the necessary data to compute $Timeline(H)$ that is the following collection:

$$\Big((p_1,T_1),(p_1,T_2),(p_1,T_3),(p_2,T_4),(p_1,T_5),(p_2,T_6)\Big)$$

The execution of the algorithm is the following:

$C \leftarrow ((p_1,T_1),(p_1,T_2),(p_1,T_3),(p_2,T_4),(p_1,T_5),(p_2,T_6))$ $Enc \leftarrow \begin{bmatrix}0\\0\end{bmatrix}$ $LastVec \leftarrow \begin{bmatrix}0\\0\end{bmatrix}$ $C \neq ()$ so we enter the while loop $(p_i, time) \leftarrow (p_1,T_1)$ $C \leftarrow ((p_1,T_2),(p_1,T_3),(p_2,T_4),(p_1,T_5),(p_2,T_6))$ $\begin{bmatrix}\alpha_1\\\alpha_2\end{bmatrix} \leftarrow LastVec = \begin{bmatrix}0\\0\end{bmatrix}$ $LastVec \leftarrow \begin{bmatrix}\neg 0\\0\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix}$ $Enc \leftarrow Enc \cdot LastVec = \begin{bmatrix}0\\0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}$ $C \neq ()$ so we enter the while loop $(p_i, time) \leftarrow (p_1,T_2)$ $C \leftarrow ((p_1,T_3),(p_2,T_4),(p_1,T_5),(p_2,T_6))$ $\begin{bmatrix}\alpha_1\\\alpha_2\end{bmatrix} \leftarrow LastVec = \begin{bmatrix}1\\0\end{bmatrix}$ $LastVec \leftarrow \begin{bmatrix}\neg 1\\0\end{bmatrix} = \begin{bmatrix}0\\0\end{bmatrix}$ $Enc \leftarrow Enc \cdot LastVec = \begin{bmatrix}0\\0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix}$ $C \neq ()$ so we enter the while loop $(p_i, time) \leftarrow (p_1,T_3)$ $C \leftarrow ((p_2,T_4),(p_1,T_5),(p_2,T_6))$ $\begin{bmatrix}\alpha_1\\\alpha_2\end{bmatrix} \leftarrow LastVec = \begin{bmatrix}0\\0\end{bmatrix}$ $LastVec \leftarrow \begin{bmatrix}\neg 0\\0\end{bmatrix} = \begin{bmatrix}1\\0\end{bmatrix}$ $Enc \leftarrow Enc \cdot LastVec = \begin{bmatrix}0\\0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}\begin{bmatrix}0\\0\end{bmatrix}\begin{bmatrix}1\\0\end{bmatrix}$

$C \neq ()$ so we enter the while loop $\quad (p_i, time) \leftarrow (p_2, T_4) \quad C \leftarrow ((p_1, T_5), (p_2, T_6)) \quad \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} \leftarrow$

$LastVec = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad LastVec \leftarrow \begin{bmatrix} 1 \\ \neg 0 \end{bmatrix} = \begin{bmatrix} 1 \\ 1 \end{bmatrix} \quad Enc \leftarrow Enc \cdot LastVec = \begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix} \; C \neq ()$

so we enter the while loop $\quad (p_i, time) \leftarrow (p_1, T_5) \quad C \leftarrow ((p_2, T_6)) \quad \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} \leftarrow LastVec = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$

$LastVec \leftarrow \begin{bmatrix} \neg 1 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \quad Enc \leftarrow Enc \cdot LastVec = \begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}$ ——————— $C \neq ()$

so we enter the while loop $\quad (p_i, time) \leftarrow (p_2, T_6) \quad C \leftarrow () \quad \begin{bmatrix} \alpha_1 \\ \alpha_2 \end{bmatrix} \leftarrow LastVec = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$

$LastVec \leftarrow \begin{bmatrix} 0 \\ \neg 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix} \quad Enc \leftarrow Enc \cdot LastVec = \begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}$ ___ $C = ()$ so we exit the while loop

Thus, the encoding of H is the word

$$\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 0 \end{bmatrix}\begin{bmatrix} 1 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 1 \end{bmatrix}\begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

$\Diamond$

Example 5. We recall the attributes of the operations:

	a	b	c
$proc$	p_1	p_1	p_2
$stime$	T_1	T_3	T_4
$rtime$	T_2	T_5	T_6
$type$	$write$	$read$	$write$
obj	y	x	x
$ival$	14	Φ	$"fly"$
$oval$	Φ	$"bee"$	Φ

Let

$$f(14) = 00 \quad f("bee") = 01 \quad f("fly") = 10$$
$$g(x) = 0 \quad g(y) = 1$$

We use colors to delimit the encodings of attributes: *types* in yellow, *values* in green and *obj* in blue.

The execution of Algorithm **??** on history H is the following (we use *wr* for *write* and *rd* for *read*).

$C \leftarrow ((p_1, T_1, wr, 14, y), (p_1, T_2, wr, 14, y), (p_1, T_3, rd, \text{"}bee\text{"}, x), (p_2, T_4, wr, \text{"}fly\text{"}, x), (p_1, T_5, rd, \text{"}bee\text{"}, x), (p_2, T_6,$

$Enc \leftarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$, $LastVec \leftarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ $C \neq ()$ so we enter the while loop $(p_i, time, type, value, obj) \leftarrow$

$(p_1, T_1, wr, 14, y)$ $C \leftarrow ((p_1, T_2, wr, 14, y), ...)$ $\begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \end{bmatrix} \leftarrow LastVec = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$ $\alpha_1 = 0$ so

$\gamma_1\gamma_2 \leftarrow f(14) = 00$ and $\lambda_1 \leftarrow g(y) = 1$ $t = 1$ because $type \neq read$ and

$$LastVec \leftarrow \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{matrix} \\ \\ t \\ \gamma_1 \\ \gamma_2 \\ \lambda_1 \end{matrix}$$

$Enc \leftarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix}$ $C \neq ()$ so we enter the while loop $(p_i, time, type, value, obj) \leftarrow (p_1, T_2, wr, 14, y)$ $C \leftarrow ((p_1, T_3, rd, \text{"}bee\text{"}, x), ...)$

$$\begin{bmatrix}\alpha_1\\ \alpha_2\\ \alpha_3\\ \alpha_4\\ \alpha_5\\ \alpha_6\end{bmatrix} \leftarrow LastVec = \begin{bmatrix}1\\ 0\\ 1\\ 0\\ 0\\ 1\end{bmatrix} \qquad \alpha_1 \neq 0 \text{ so}$$

$$LastVec \leftarrow \begin{bmatrix}0\\ 0\\ 0\\ 0\\ 0\\ 0\end{bmatrix}$$

$$Enc \leftarrow \begin{bmatrix}0\\ 0\\ 0\\ 0\\ 0\\ 0\end{bmatrix}\begin{bmatrix}1\\ 0\\ 1\\ 0\\ 0\\ 1\end{bmatrix}\begin{bmatrix}0\\ 0\\ 0\\ 0\\ 0\\ 0\end{bmatrix} \text{_____} C \neq () \text{ so we enter the}$$

while loop $\quad (p_i, time, type, value, obj) \leftarrow (p_1, T_3, rd, "bee", x) \qquad C \leftarrow ((p_2, T_4, wr, "fly", x), ...)$

$$\begin{bmatrix}\alpha_1\\ \alpha_2\\ \alpha_3\\ \alpha_4\\ \alpha_5\\ \alpha_6\end{bmatrix} \leftarrow LastVec = \begin{bmatrix}0\\ 0\\ 0\\ 0\\ 0\\ 0\end{bmatrix} \qquad \alpha_1 = 0 \text{ so } \gamma_1\gamma_2 \leftarrow f("bee") = 01 \text{ and } \lambda_1 \leftarrow g(x) = 0 \qquad t = 0 \text{ because}$$

$type = read$ and

$$LastVec \leftarrow \begin{array}{c} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \end{array} \begin{array}{l} \\ \\ t \\ \gamma_1 \\ \gamma_2 \\ \lambda_1 \end{array}$$

$$Enc \leftarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$C \neq ()$ so we enter the while loop $(p_i, time, type, value, obj) \leftarrow (p_2, T_4, wr, "fly", x)$ $C \leftarrow ((p_1, T_5, rd, "bee", x), ...)$

$$\begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \end{bmatrix} \leftarrow LastVec = \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix}$$

$\alpha_2 = 0$ so $\gamma_1\gamma_2 \leftarrow f("fly") = 10$ and $\lambda_1 \leftarrow g(x) = 0$ $t = 1$ because $type \neq read$ and

$$LastVec \leftarrow \begin{array}{c} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \end{array} \begin{array}{l} \\ \\ t \\ \gamma_1 \\ \gamma_2 \\ \lambda_1 \end{array}$$

$$Enc \leftarrow \begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\1\\0\\0\\1\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\\0\\1\\0\end{bmatrix}\begin{bmatrix}1\\1\\1\\1\\0\\0\end{bmatrix}$$ $C \neq ()$ so we enter the while loop $(p_i, time, type, value, obj) \leftarrow (p_1, T_5, rd, "bee", x)$ $C \leftarrow ((p_2, T_6, wr, "fly", x))$

$$\begin{bmatrix}\alpha_1\\\alpha_2\\\alpha_3\\\alpha_4\\\alpha_5\\\alpha_6\end{bmatrix} \leftarrow LastVec = \begin{bmatrix}1\\1\\1\\1\\0\\0\end{bmatrix}$$ $\alpha_1 = 1$ so and

$$LastVec \leftarrow \begin{bmatrix}0\\1\\0\\0\\0\\0\end{bmatrix}$$

$$Enc \leftarrow \begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\1\\0\\0\\1\end{bmatrix}\begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix}\begin{bmatrix}1\\0\\0\\0\\1\\0\end{bmatrix}\begin{bmatrix}1\\1\\1\\1\\0\\0\end{bmatrix}\begin{bmatrix}0\\1\\0\\0\\0\\0\end{bmatrix}$$ $C \neq ()$ so we enter the while

loop $(p_i, time, type, value, obj) \leftarrow (p_2, T_6, wr, "fly", x)$ $C \leftarrow ()$ $\begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \alpha_3 \\ \alpha_4 \\ \alpha_5 \\ \alpha_6 \end{bmatrix} \leftarrow LastVec = \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$

$\alpha_2 = 1$ so and

$$LastVec \leftarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$$Enc \leftarrow \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

$C = ()$ so we exit the while loop

Thus, the encoding of H with types, values and objects is

$$\begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 1 \\ 0 \\ 0 \\ 1 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

We highlight that data about types/values/object is only encoded on the first vector of an operation. Here for a, b and c.

$$\begin{array}{ccccccc} & \downarrow & & \downarrow & \downarrow & & \\ \begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix} & \begin{bmatrix}1\\0\\1\\0\\0\\1\end{bmatrix} & \begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix} & \begin{bmatrix}1\\0\\0\\0\\1\\0\end{bmatrix} & \begin{bmatrix}1\\1\\1\\1\\0\\0\end{bmatrix} & \begin{bmatrix}0\\1\\0\\0\\0\\0\end{bmatrix} & \begin{bmatrix}0\\0\\0\\0\\0\\0\end{bmatrix} \end{array}$$

$\Diamond$